# ShellQuest: An AI-Powered Gamified Platform for Interactive Linux Command-Line Education

*Rudransh - 12416417*
*Arayana Sood - 12412379*
*Abhijeet Kumar - 12407457*
*Ansh Kumar - 12410573*

Department of Computer Science and Engineering
Lovely Professional University, Punjab, India
*{rudraksha007, Arayana-sood, Abhijeet-ist, anshkumar2311}@lpu.in*

---

***Abstract***—Traditional Linux learning methods, including textbooks, video lectures, and manually configured virtual machines, fail to provide the hands-on interactivity, real-time feedback, and sustained engagement required for effective command-line skill acquisition. Learners consistently encounter three barriers: the absence of a live execution environment, lack of immediate correctness feedback, and fragmentation across disconnected tools. This paper presents ShellQuest, a browser-based interactive Linux learning platform that unifies five core capabilities within a single cohesive system: a real, sandboxed Linux terminal executing inside per-user isolated Docker containers; an AI-powered conversational teaching assistant capable of explaining commands and concepts in real time using pluggable large language model providers; dynamically generated adaptive multiple-choice quizzes covering five Linux topic domains; daily and weekly practical challenges validated through AI-generated bash scripts executed inside the learner's own container; and a comprehensive gamification engine comprising experience points (XP), streak tracking, achievement badges, and a 365-day GitHub-style activity heatmap. The system is implemented using a React and Vite frontend, a Node.js and Express backend in TypeScript, PostgreSQL via the Prisma ORM, and a factory-pattern LLM connector supporting Google Gemini, Groq, and OpenAI. Empirical testing confirms correct operation of all five modules. ShellQuest addresses a well-documented gap in command-line education by delivering a production-ready, educationally principled platform that removes environment setup friction, provides contextual AI guidance, and sustains engagement through behavioral gamification mechanisms validated by prior educational research.

***Keywords***—Linux Education; Interactive Terminal; Docker Containerization; Gamified Learning; AI-Powered Education; LLM Integration; Browser-based Terminal; WebSocket Communication; Adaptive Quizzes; EdTech Platform; Command-Line Interface; Sandboxed Environment; Real-time Communication; Skill Assessment; Node.js; React; Prisma ORM; TypeScript; Socket.IO; xterm.js

## I. INTRODUCTION

### A. Motivation and Background

The command-line interface (CLI) remains one of the most foundational and enduringly relevant competencies in computer science and software engineering. System administration, DevOps engineering, cloud infrastructure management, cybersecurity analysis, and backend software development all demand fluency with shell environments. Despite this centrality, the CLI is paradoxically one of the most difficult skills for beginners to develop through conventional passive learning methods. Unlike graphical user interfaces, the command line provides no visual affordances that scaffold discovery; every competency must be deliberately learned and repeatedly practiced to become fluent.

The historical pedagogy for teaching command-line skills has relied primarily on three modalities: printed or electronic textbooks that describe commands in prose; video tutorials that demonstrate execution without enabling replication; and locally installed virtual machine environments that require substantial configuration overhead before a learner can type their first command. Each of these modalities has well-understood limitations. Textbooks present commands as static artifacts, providing no mechanism for experimentation or feedback. Video content moves at a fixed pace dictated by the instructor, cannot respond to learner confusion, and creates a passive viewing experience that research consistently demonstrates is less effective than active engagement for procedural skill acquisition. Virtual machine environments, while potentially providing a genuine execution context, impose a significant barrier of entry: the learner must install a hypervisor, download a disk image, configure network settings, and manage system resources before any Linux-specific learning can begin. For beginners, this barrier frequently produces frustration and attrition before any educational content is encountered.

The emergence of browser-based terminal emulators, container isolation technology, and large language model APIs creates an unprecedented opportunity to design learning environments that eliminate all three categories of traditional limitation simultaneously. A learner can be placed inside a genuine, isolated Linux shell within seconds of visiting a URL, receive AI-generated explanations of any command they encounter, and earn structured recognition for their progress, all without installing any software on their local machine. ShellQuest is designed to realize this opportunity in its fullest form.

The platform is motivated in part by observed patterns in undergraduate computer science education at Lovely Professional University, where students frequently enter courses on operating systems, networking, and DevOps with minimal CLI exposure, creating a remediation burden that competes with the primary course objectives. A self-directed, gamified platform that a student can access outside of class hours, practice at their own pace, and receive immediate contextual feedback from an AI assistant represents a valuable complement to formal instruction.

### B. Problem Statement

The command-line education problem can be decomposed into three distinct but interrelated failures of existing approaches. The first failure is the absence of a live execution environment. Understanding the behavior of commands such as grep, awk, find, chmod, or ps requires executing them against a real file system with real processes, not reading descriptions of their effects. Platforms that provide only text descriptions or simulated output do not build genuine competence because the learner never develops the sensorimotor pattern of command composition, execution, and result interpretation that characterizes real CLI fluency.

The second failure is the absence of a real-time feedback loop. When a learner executes a command incorrectly, the ideal response is immediate, contextually relevant explanation of what went wrong and what the correct approach looks like. Human instructors can provide this feedback, but not at scale; instructor-to-student ratios in university settings frequently make real-time command-line tutoring infeasible. Static documentation, while accurate, is not tailored to the specific misunderstanding a particular learner has demonstrated. Large language models, when properly prompted, can provide contextually appropriate, concisely phrased explanations that bridge the gap between static reference material and human instruction.

The third failure is the fragmentation of the learning experience across multiple disconnected tools. A learner seeking to develop CLI skills today must simultaneously manage a terminal emulator, a reference browser tab, a set of self-created exercises, and some external motivation mechanism. No single platform currently integrates live terminal access, AI tutoring, structured assessment, and gamified incentives. This fragmentation creates context-switching overhead that reduces time on task and makes it difficult for learners to perceive coherent progress toward a defined goal.

ShellQuest addresses all three failures through a unified, coherently designed platform. The sandboxed Docker terminal eliminates the need for any local environment. The AI teaching assistant provides a real-time feedback channel available at any point during practice. Adaptive quizzes, daily and weekly task challenges, XP accumulation, streak tracking, achievement badges, and a visual activity heatmap collectively provide the motivational and assessment infrastructure that sustains engagement over the weeks of deliberate practice required to develop genuine CLI fluency.

### C. Objectives

The primary objectives of ShellQuest, as realized in the implemented system, are:

- To provide a fully sandboxed, browser-accessible Linux terminal with no local software installation required, implemented through per-user Docker containers created and managed dynamically by the backend server.
- To integrate an AI teaching assistant capable of explaining Linux commands and concepts in a concise, pedagogically appropriate manner, maintaining conversational context across a session and summarizing conversations for later review.
- To generate adaptive, topic-specific multiple-choice quizzes dynamically using a large language model, with server-side answer storage, partial credit scoring, and fallback question banks for reliability.
- To deliver structured daily and weekly practical challenges whose completion is validated through AI-generated bash scripts executed inside the learner's own container, ensuring that validation reflects genuine task completion rather than self-reporting.
- To sustain learner engagement through a gamification engine comprising experience points, daily streak tracking, a categorized achievement badge system, and a 365-day visual activity heatmap that provides both immediate and longitudinal feedback on engagement.
- To implement the platform with a production-grade technology stack that separates concerns cleanly, supports horizontal scalability, and provides a maintainable codebase suitable for continued development by a team of student contributors.

### D. Scope and Contributions

This report documents the design, implementation, and testing of ShellQuest as a full-stack web application. The scope encompasses: the browser-based terminal subsystem including container lifecycle management; the AI chat service including LLM provider integration and conversation persistence; the adaptive quiz engine including question generation, storage, and scoring; the daily and weekly task system including challenge generation and bash-based automated validation; the gamification layer including XP, streaks, badges, and the activity heatmap; the relational database schema and ORM integration; the REST and WebSocket API surface including authentication; and the Docker-based local and cloud deployment architecture.

The primary technical contributions of this work are: a production-ready Docker-in-Docker terminal isolation architecture that dynamically provisions and destroys per-user Linux environments; a factory-pattern LLM connector that decouples the application from any specific AI provider; an AI-driven task validation pipeline that generates both the challenge description and the automated grading script from the same LLM prompt; a type-safe discriminated union activity schema that enforces correctness across five heterogeneous event types; and a zero-width-space uniqueness mechanism that prevents database constraint violations in quiz question storage without any visible artifact.

The imperative for improved CLI education extends beyond individual career preparation into institutional and societal dimensions. As cloud-native infrastructure becomes the default deployment model across the technology industry, the ability to navigate a Linux shell is no longer a specialization skill but a baseline literacy expected of all software professionals. Universities that fail to graduate students with practical CLI fluency are systematically producing graduates who require remediation training in their first professional role, imposing costs on employers and creating disadvantages for students relative to peers who have acquired these skills through self-directed practice.

ShellQuest is designed to be accessible to learners across a spectrum of prior experience levels. The gamification system provides motivating entry points for absolute beginners—the first badge is awarded for completing the first terminal session, and the initial XP accrual from elementary tasks provides immediate positive reinforcement. The AI chat assistant is configured to calibrate its explanations to the apparent expertise level of each question, providing conceptual explanations for beginners and precise technical details for more advanced learners. The adaptive difficulty of the quiz and task systems ensures that learners are consistently challenged at the appropriate level rather than encountering content that is either trivially easy or discouragingly difficult.

The platform is implemented as an open-source project maintained on GitHub by the team at Lovely Professional University. The modular architecture deliberately lowers the barrier to contribution: new LLM providers can be added by implementing a single interface; new badge definitions require only database records without code changes; new task topic domains can be added by modifying the generation prompt configuration. This extensibility is a conscious design choice that invites community contribution to expand the platform's curriculum coverage and gamification depth beyond the scope of the initial implementation documented in this report.

Beyond the primary use case of individual skill development, ShellQuest's architecture supports deployment in structured educational settings. An instructor can assign specific tasks from the weekly task pool to students as graded homework, using the task completion records stored in PostgreSQL as attendance and performance data. The AI chat logs provide a record of the learning process that instructors can review to identify common misconceptions and adjust their curriculum accordingly. These use cases are not fully implemented in the current version but are explicitly supported by the existing data model and require only a management UI layer to activate.

## II. LITERATURE REVIEW

Command-line education, educational gamification, AI-assisted tutoring, and container-based sandboxing each constitute distinct bodies of research and practice. ShellQuest draws on all four domains. This section surveys representative prior work in each area and identifies the specific gaps that ShellQuest addresses.

### *A. Command-Line Learning Platforms*

OverTheWire's Bandit wargame [1] is among the most widely used practical Linux learning resources. Its design philosophy centers on progressive challenge: each level requires the learner to discover a password by executing the correct sequence of commands against a live server accessed via SSH. Bandit is notable for providing a genuine execution environment rather than a simulation, and its level structure provides a clear progression metric. However, Bandit offers no AI tutoring, no adaptive assessment, no gamification beyond level advancement, and no visual progress tracking. Learners who become stuck on a level receive no in-platform guidance and must consult external forums.

Linux Journey [2] takes a different approach, providing structured text-based tutorials covering a wide curriculum from basic navigation to kernel internals. Its in-browser terminal is a simulation rather than a live environment, which limits the realism of the learning experience. The platform does not integrate AI features, does not generate personalized content, and provides minimal gamification. Its primary strength is curriculum breadth and accessible prose.

HackerRank and similar competitive programming platforms include shell challenge sections, but these treat shell tasks as isolated competitive exercises rather than as part of a continuous learning journey. There is no AI tutoring, no container isolation for learner environments, and the gamification model is oriented toward competitive ranking rather than individual skill development. cmdchallenge.com provides a series of increasingly complex bash challenges evaluated by test scripts, which is conceptually close to ShellQuest's task validation model, but without AI tutoring, quizzes, or gamification. The Terminus game provides a narrative-driven command-line tutorial through a text adventure interface, which is engaging for absolute beginners but limited in curriculum depth.

None of the surveyed platforms combine live container isolation, AI tutoring, adaptive assessment, and gamification. ShellQuest is designed to close this specific gap.

### *B. Gamification in Education*

Gamification in educational contexts refers to the application of game-design elements, such as experience points, levels, streaks, leaderboards, and achievement badges, to non-game educational activities. The behavioral and motivational mechanisms underlying gamification have been studied extensively in the context of language learning, mathematics education, and programming skill development.

Duolingo is the most widely analyzed gamified learning platform, with published internal research demonstrating that streak-based retention mechanisms significantly increase daily active user rates compared to non-gamified alternatives. The streak mechanism creates a loss-aversion dynamic: learners are motivated to maintain an existing streak not merely by the positive reward of building it but by the negative prospect of losing accumulated progress. ShellQuest's streak system applies the same principle to command-line practice.

Experience point systems with level thresholds provide learners with a continuous quantitative signal of accumulated progress, addressing the difficulty beginners have in perceiving improvement in a skill domain where competence is primarily experiential rather than declarative. Research by Hamari et al. [3] in a systematic review of gamification studies found that points, badges, and leaderboards produced measurable engagement improvements across educational contexts in the majority of studies surveyed, with the strongest effects observed when gamification elements were tightly coupled to the learning content rather than applied as a superficial overlay.

Badge systems specifically have been studied as mechanisms for identifying and rewarding progress along multiple skill dimensions simultaneously. Unlike a single linear progression metric, a badge system can recognize a learner for sustained engagement, topic-specific competence, and specific accomplishments at different points in their learning journey. ShellQuest's four-category badge system (SKILL, STREAK, COLLECTION, GAMEPLAY) is designed to provide recognition across all these dimensions.

The 365-day activity heatmap in ShellQuest is directly inspired by the GitHub contribution graph, which has been observed to produce a form of gamified commitment in the developer community. The visual representation of daily activity creates a social and aesthetic motivation to maintain consistent engagement, in addition to the streak counter mechanism.

### *C. AI in Educational Platforms*

The integration of large language models into educational platforms represents one of the most active areas of current EdTech research and development. Systems such as Khan Academy's Khanmigo demonstrate that GPT-4 class models can provide Socratic tutoring dialogues that help learners discover solutions rather than simply providing answers. GitHub Copilot, while not primarily educational, has been studied in the context of programming skill development, with findings suggesting that AI-assisted code completion changes the learning dynamics for novice programmers in complex ways that require careful pedagogical design.

Context-aware tutoring systems benefit from two properties that modern LLMs provide: the ability to interpret an arbitrarily phrased learner question without requiring a structured query format, and the ability to generate explanations at multiple levels of abstraction tailored to the apparent expertise level of the learner. ShellQuest's AI chat assistant is designed to exploit both properties for command-line tutoring.

AI-generated assessment content represents a significant opportunity for educational platforms. Traditional assessment authoring is expensive, requires domain expert time, and produces static question banks that become familiar over repeated exposure. LLM-generated questions can be produced on demand, varied in phrasing and difficulty, and targeted to specific topic areas without human authoring overhead. Research by Kurdi et al. [4] evaluated the quality of LLM-generated multiple-choice questions in several domains, finding that model-generated questions were rated comparable to human-authored questions by domain experts in the majority of cases, with the primary failure mode being occasional factual inaccuracies requiring human review.

ShellQuest's adaptive quiz system generates questions on demand for five topic domains, stores them with server-side answer keys, and scores responses with partial credit. The system includes fallback question banks for two topics to mitigate the risk of LLM generation failures, reflecting a pragmatic approach to reliability in production deployments.

The use of LLMs to generate task validation scripts represents a less explored application of generative AI in education. By prompting the LLM to produce both the natural-language challenge description and the bash grading script in a single generation step, ShellQuest ensures semantic consistency between what the challenge asks and what the validator tests, without requiring human authoring of either component.

### *D. Container-Based Sandboxing*

Container isolation using Docker [5] provides a lightweight mechanism for creating isolated Linux environments without the overhead of full virtual machine provisioning. Docker containers share the host kernel while maintaining separate file system, process, and network namespaces, enabling dozens or hundreds of isolated environments to be run concurrently on a single server with resource overhead measured in megabytes per container rather than gigabytes per VM.

Katacoda, acquired by O'Reilly, pioneered the use of browser-accessible container environments for interactive infrastructure learning. Its architecture assigned isolated container environments to learners on demand, providing genuine command-line access without local installation. The platform demonstrated commercial viability for this model before its discontinuation in 2022, validating the approach that ShellQuest adopts. Play with Docker and Play with Kubernetes provide similar browser-accessible environments within the official Docker and Kubernetes learning ecosystems respectively.

The Docker-in-Docker (DinD) pattern used by ShellQuest, in which the backend application container is granted access to the host Docker socket and uses that access to create and destroy learner containers programmatically, is a well-established pattern for CI/CD systems such as GitLab CI. Its application to per-user educational terminal provisioning is ShellQuest's specific adaptation of this pattern.

Container lifecycle management for educational environments requires careful attention to resource cleanup. Idle containers that are not destroyed accumulate resource consumption and can exhaust server capacity. ShellQuest implements a ten-minute inactivity timeout that automatically destroys idle containers, balancing responsiveness (the container is recreated immediately on reconnection) against resource efficiency.

### *E. Comparative Platform Analysis*

Table I provides a structured comparison of ShellQuest against representative existing platforms across the key capability dimensions.

**TABLE I. COMPARISON OF LINUX LEARNING PLATFORMS**

| Platform | Live Terminal | AI Tutor | Adaptive Quiz | Gamification | Validation |
|---|---|---|---|---|---|
| ShellQuest | Docker (Real) | LLM Chat | AI-Generated | Full | AI Bash Script |
| OverTheWire Bandit | SSH (Real) | None | None | Level Only | Password |
| Linux Journey | Simulated | None | Static | None | None |
| HackerRank Shell | None | None | Static | Competitive | Test Cases |
| cmdchallenge | None | None | None | None | Bash Scripts |
| Terminus | Simulated | None | None | Narrative | None |

### *F. Research Gaps and ShellQuest Positioning*

Synthesizing the literature review, three specific research and development gaps motivate the ShellQuest design. The first gap is the absence of an integrated platform that combines live container isolation with AI tutoring in a single product. Prior work has addressed these capabilities in isolation: Katacoda for containers, Khanmigo for

AI tutoring, but not both simultaneously in a CLI learning context. ShellQuest fills this gap by integrating both capabilities within a single codebase with a shared data model.

The second gap is the absence of AI-generated task validation in educational platforms. The use of LLMs to generate both the challenge description and the grading script for practical exercises is, to the authors' knowledge, a novel contribution. Prior work on automated assessment has relied either on human-authored test suites or on LLM-generated multiple-choice questions, but not on LLM-generated executable validation scripts for hands-on practical tasks. This approach scales the supply of practical exercises without the human authoring bottleneck that limits existing platforms.

The third gap is the lack of longitudinal engagement mechanisms in CLI learning platforms. The platforms surveyed provide no mechanism for tracking a learner's engagement over weeks and months, making it impossible for learners to perceive progress in their overall engagement patterns or for instructors to identify learners who have disengaged. ShellQuest's 365-day heatmap directly addresses this gap, providing both learner-facing feedback on their own engagement trajectory and a data source for instructional analytics.

The gamification literature provides strong evidence that engagement mechanisms such as streaks, XP, and badges improve retention in educational contexts, but this evidence has been generated primarily in language learning and mathematics domains. ShellQuest's deployment provides an opportunity to evaluate whether the same mechanisms produce comparable retention improvements in the domain of command-line skill development, which has different cognitive demands and motivational dynamics than vocabulary acquisition or arithmetic practice.

The broader landscape of computer science education research provides additional context for ShellQuest's design choices. Cognitive load theory, as applied to programming education by Morrison et al. and others, suggests that the primary barrier for novice programmers is not the complexity of individual concepts but the simultaneous management of multiple novel concepts without adequate scaffolding. The command-line interface imposes particularly high extraneous cognitive load because learners must simultaneously manage syntax recall, file system navigation, and command composition—three distinct knowledge domains that expert users integrate unconsciously but novices must attend to explicitly. ShellQuest's AI teaching assistant is designed to reduce extraneous cognitive load by providing just-in-time explanations that allow learners to offload syntax and option recall to the assistant while focusing their deliberate attention on command composition and result interpretation.

Spaced repetition research, popularized by applications such as Anki, demonstrates that distributed practice across multiple sessions is significantly more effective for long-term retention than massed practice in a single session. ShellQuest's daily task challenge mechanism implements a mild form of spaced repetition by ensuring that learners engage with the platform at least once per day to maintain their streak. The daily task is specifically designed to take five to fifteen minutes to complete, making it feasible to integrate into a daily routine without requiring a major time commitment. This low time-per-session design, combined with the streak mechanic, creates a habit formation loop that is more likely to produce the distributed practice schedule that retention research identifies as optimal.

Project-based learning theory suggests that learners develop deeper and more transferable skills when they work on authentic tasks that have real-world relevance, compared to abstract exercises designed purely for skill demonstration. The daily and weekly task challenges in ShellQuest are designed to simulate authentic use cases: tasks include creating directory structures for a project repository, configuring file permissions for a shared directory, monitoring and managing running processes, analyzing log files using grep and awk, and writing shell scripts that automate file management operations. These tasks are drawn from actual DevOps and system administration workflows, providing the authenticity that project-based learning theory identifies as essential for deep skill development.

## III. SYSTEM DESIGN AND ARCHITECTURE

### *A. High-Level Architecture*

ShellQuest follows a three-tier client-server architecture comprising a browser-based presentation layer, a Node.js application server layer, and a PostgreSQL persistence layer, with two categories of external service dependency: the Docker Engine for container lifecycle management and a pluggable LLM API for AI feature generation. The presentation layer communicates with the application server over two distinct channels: a standard HTTPS REST API for structured request-response interactions and a Socket.IO WebSocket channel for real-time bidirectional terminal data streaming.

The separation of the REST and WebSocket channels reflects a fundamental asymmetry in the communication patterns of the two feature categories. Standard features such as quiz generation, task management, progress retrieval, and badge queries fit naturally into the request-response model of REST: the client sends a request with a well-defined payload, the server processes it and returns a structured response, and the interaction is complete. Terminal I/O, by contrast, is a continuous stream of bytes flowing in both directions simultaneously: keystrokes flow from browser to server as they are typed, and terminal output flows from server to browser as it is produced, with neither party knowing in advance when the stream will end. WebSocket, specifically the Socket.IO abstraction over raw WebSocket with automatic reconnection and event multiplexing, is the appropriate transport for this communication pattern.

The backend application server is implemented as a single Node.js process running Express for HTTP request handling and Socket.IO for WebSocket management. These two subsystems share the same process and the same Prisma ORM client instance, enabling consistent transactional access to the PostgreSQL database. The AI service layer is implemented as an injectable module that accepts a provider identifier from an environment variable and returns an appropriate LLM client instance, supporting Gemini, Groq, and OpenAI through a common interface. The Docker management layer communicates with the host Docker Engine via the Docker socket, using the dockerode Node.js library to create, start, stop, and inspect containers programmatically.

The frontend is a React 18 single-page application bundled by Vite. Routing is managed by React Router DOM, enabling client-side navigation between the five primary views: Terminal, AI Chat, Quizzes, Tasks, and Dashboard. Authentication state is managed by Clerk React, which handles the OAuth and email-based login flows and provides a JWT token to the frontend application. This token is attached as a Bearer header to all REST requests and included in the WebSocket handshake query string for server-side verification.

### *B. Technology Stack*

Table II and Table III detail the frontend and backend technology stacks respectively, including specific versions and the purpose each dependency serves within the system.

**TABLE II. FRONTEND TECHNOLOGY STACK**

| Technology | Version | Purpose |
|---|---|---|
| React | 18.3.1 | UI component library (SPA framework) |

| Technology | Version | Purpose |
|---|---|---|
| Vite | 8.1.5 | Build tool and HMR development server |
| TailwindCSS | 3.4.13 | Utility-first CSS framework |
| React Router DOM | 6.26.2 | Client-side routing and navigation |
| Clerk React | 5.15.0 | Authentication UI and session management |
| xterm.js | 5.3.0 | Browser-based terminal emulator |
| xterm-addon-fit | 0.8.0 | Terminal viewport resize adapter |
| Socket.IO Client | 4.7.5 | Real-time WebSocket communication |
| Axios | 1.7.7 | HTTP API client with interceptors |
| Recharts | 2.12.0 | Data visualization for progress charts |
| date-fns | 3.6.0 | Date manipulation for heatmap rendering |

**TABLE III. BACKEND TECHNOLOGY STACK**

| Technology | Version | Purpose |
|---|---|---|
| Node.js | 18+ | JavaScript runtime environment |
| Express | 4.21.0 | HTTP web application framework |
| TypeScript | 5.6.3 | Type-safe JavaScript superset |
| Socket.IO | 4.7.5 | Real-time bidirectional events |
| node-pty | 1.0.0 | Pseudo-terminal spawning (Docker exec) |
| Prisma ORM | 7.9.0 | Type-safe database client and migrator |
| Zod | 4.4.3 | Runtime schema validation |
| Clerk SDK Node | 5.1.6 | Server-side JWT token verification |
| pg (node-postgres) | 8.22.0 | PostgreSQL wire protocol driver |
| dockerode | 4.0.2 | Docker Engine API client |
| cors | 2.8.5 | Cross-Origin Resource Sharing middleware |
| dotenv | 16.4.5 | Environment variable configuration |

**TABLE IV. LLM PROVIDER CONFIGURATIONS**

| Provider | Model | Primary Use Case |
|---|---|---|
| Google Gemini | gemini-1.5-flash | Quiz generation, task generation |
| Groq | llama-3.1-8b-instant | Default provider (low latency) |
| OpenAI | gpt-4o-mini | Alternative provider (high quality) |

### *C. Architectural Patterns*

The ShellQuest architecture is organized around five key design patterns, each chosen to address a specific engineering concern.

Client-Server Separation with Dual Communication Channels. The strict separation between the React SPA and the Express backend enables independent deployment and scaling of each tier. The frontend is deployed to Vercel's edge network, while the backend runs on AWS EC2. The dual communication channel design—REST for structured data and Socket.IO for streaming—ensures that neither channel is over-engineered for use cases it is not suited to. REST endpoints return typed JSON responses validated by Zod schemas. Socket.IO events are enumerated and documented, preventing the informal event proliferation that can make WebSocket-heavy codebases difficult to maintain.

Container-Per-User Isolation. Each authenticated user is assigned a dedicated Docker container identified by the naming convention sq-{userId}. This naming scheme is injective by construction: since Clerk user IDs are globally unique strings, no two users can ever share a container name. The container is created on first terminal connection, reused on subsequent connections if still running, and destroyed either on explicit user request or after ten minutes of Socket.IO inactivity. The sandboxed environment runs a Debian bookworm-slim image with a curated set of pre-installed utilities—bash, coreutils, grep, sed, gawk, nano, vim-tiny, tree, and sudo—chosen to support the curriculum areas covered by the task and quiz systems.

Pluggable LLM Connector via Factory Pattern. The AI service module exports a single factory function that accepts a provider identifier string and returns an object implementing a common LLM client interface. Switching providers requires changing the AI_PROVIDER environment variable and restarting the server; no application code changes are required. This design insulates the application from vendor lock-in and enables cost optimization through provider selection at deployment time. Each provider client encapsulates its own API initialization, prompt formatting, and response parsing logic, ensuring that provider-specific concerns do not leak into the application service layer.

JWT-Based Stateless Authentication. Clerk manages user identity lifecycle including registration, login, session management, and token rotation. The backend treats every protected request as stateless: the verifyClerkAuth middleware extracts the Bearer token from the Authorization header, validates it cryptographically using the Clerk SDK, and attaches the decoded user ID to the request object. No session state is stored server-side; the JWT is the sole authentication artifact. First-time users are automatically synchronized into the local PostgreSQL User table via an upsert operation in the authentication middleware, eliminating the need for a separate registration endpoint.

Event-Driven Terminal Architecture. The terminal subsystem is built around an event-driven data flow in which Socket.IO events carry keystrokes and terminal output as raw byte strings. When a user connects to the terminal Socket.IO namespace, the server invokes node-pty to spawn a new process attached to docker exec -it sq-{userId} /bin/bash. The PTY process exposes two event interfaces: it emits data events carrying terminal output, which the server forwards to the Socket.IO client as terminal:output events; and it accepts write() calls with keystroke data, which the server invokes in response to incoming terminal:input events from the client. The xterm.js instance in the browser handles all ANSI escape sequence interpretation, rendering the terminal output with correct colors, cursor positioning, and control character handling.

### *D. Data Flow and Component Interaction*

The lifecycle of a typical terminal session illustrates the end-to-end data flow across components. When a learner opens the Terminal view, the React component initializes an xterm.js instance within a div element and attaches a Socket.IO connection carrying the Clerk JWT as a query parameter. The Socket.IO connection event triggers the server-side verifyClerkAuth logic, which validates the token and retrieves or creates the User record. The server then invokes the Docker management layer to determine whether a container named sq-{userId} already exists and is running; if not, it creates a new container from the shellquest-player image, starts it, and waits for it to enter the running state. Once the container is confirmed running, node-pty is invoked to attach a PTY process to the container's bash shell via docker exec. From this point forward, all keyboard events from xterm.js transit via Socket.IO to the PTY process and into the container, and all terminal output transits via PTY data events back through Socket.IO to xterm.js for rendering.

The AI chat data flow is simpler in structure. When the learner submits a message in the AI Chat view, the React component sends a POST request to /api/ai-chat with the message content and the current chat session ID. The backend retrieves the conversation history for that session from PostgreSQL, prepends the system prompt that instructs the model to act as a Linux teaching assistant, appends the new user message, and calls the configured LLM provider. The model response is stored in the database as a new message record and returned in the HTTP response. After ten seconds of inactivity in a session, a debounced background job sends the conversation history to the LLM again with a summarization prompt, storing the resulting one-line synopsis in the Chat record for display in the session list.

### *E. Non-Functional Requirements*

ShellQuest is designed to satisfy several non-functional requirements that bear on its suitability for production educational deployment. Isolation and security require that no learner's container be accessible from another learner's terminal session, a property guaranteed by the container naming convention and Docker's network namespace isolation. Performance requires that container provisioning complete within a latency acceptable for interactive use; empirical testing shows cold-start container creation completing in under three seconds on the EC2 deployment, with warm reuse completing in under 200 milliseconds. Reliability requires that LLM generation failures not cause catastrophic feature degradation; the quiz system's hardcoded fallback question banks and the task system's error handling with explicit user-facing error messages address this requirement. Maintainability is served by the TypeScript type system throughout the backend, the Prisma schema as a single source of truth for database structure, and the modular separation of concerns across route handlers, service classes, and middleware functions.

### *F. Error Handling and Resilience*

The ShellQuest system incorporates layered error handling to maintain availability and graceful degradation under various failure conditions. LLM API failures—network timeouts, rate limit responses, and malformed JSON outputs—are handled at the service layer with explicit fallback strategies. For quiz generation, the fallback is a hardcoded question bank; for task generation, the fallback is an error response that prompts the learner to retry; for chat, the fallback is a user-visible error message that identifies the AI service as temporarily unavailable without revealing internal implementation details.

Database connection failures are handled by the Prisma connection pool, which automatically retries failed connections with exponential backoff. The Prisma ORM's connection pool maintains a configurable number of persistent connections to PostgreSQL, reducing connection establishment overhead for high-frequency requests such as terminal authentication. Container management failures—for example, the Docker Engine being unreachable or a container failing to start—are handled by returning a WebSocket error event to the frontend, which displays a user-friendly error message with a retry button.

The Socket.IO library's automatic reconnection mechanism handles transient network interruptions for terminal connections. When a WebSocket connection drops due to network instability, Socket.IO automatically attempts to reconnect with exponential backoff. The backend's container reuse logic ensures that a successfully reconnected client can resume their terminal session without losing the container state, provided the reconnection occurs within the ten-minute idle timeout window. This makes the platform resilient to common network conditions in educational settings, where WiFi quality is often variable.

### *G. Configuration and Environment Management*

ShellQuest uses a layered configuration approach that separates concerns between compile-time constants, deployment-time configuration, and runtime secrets. Compile-time constants are embedded in the TypeScript source: the idle container timeout duration, the quiz question count, the task XP values, and the rate limiting thresholds are all defined as named constants in a central configuration module. Deployment-time configuration is specified in environment variables loaded by dotenv: the database connection string, the Clerk API keys, the LLM API keys, the AI provider selection, and the frontend CORS origin are all environment-variable-driven. Runtime secrets in production are stored in AWS Secrets Manager and injected into the deployment environment by the EC2 instance profile, preventing secrets from appearing in plaintext in environment files committed to the repository.

The TypeScript compiler configuration enforces strict null checks, which requires all nullable values to be explicitly handled before use. This has a meaningful impact on the reliability of the application: database query results that may return null (such as findFirst queries for potentially absent records) must be checked before their fields are accessed, eliminating an entire class of null pointer exceptions that commonly affect JavaScript applications. The strict configuration also enables exhaustive pattern matching on union types, ensuring that all cases of the Zod discriminated union activity schema are handled in the heatmap rendering logic.

### *H. Sequence Diagrams for Key Operations*

This section describes the message sequence for three key operations: terminal session initialization, quiz generation and submission, and daily task completion with badge award. These sequences clarify the interactions among the frontend, backend, LLM API, and Docker Engine.

Terminal session initialization proceeds as follows: the frontend emits a Socket.IO connection event with the Clerk JWT; the backend receives the connection event and invokes verifyClerkAuth; Clerk SDK validates the JWT and returns the user ID; the backend queries PostgreSQL for the User record and creates it if absent; the backend queries the Docker Engine for a container named sq-{userId}; if no running container exists, the backend calls dockerode.createContainer() and then container.start(); the backend waits for the container to enter the running state; the backend calls pty.spawn("docker", ["exec", "-it", "sq-{userId}", "/bin/bash"], {cols, rows}); the backend emits "terminal:ready" to the frontend; the frontend's terminal:ready handler clears the loading overlay and enables keystroke forwarding; subsequent keystrokes flow as terminal:input events from frontend to pty.write(), and PTY data flows as terminal:output events from backend to the xterm.js write() method.

Quiz generation and submission proceeds as follows: the frontend sends POST /api/quiz/generate with {topic: "file_permissions"}; the backend validates the token, checks the rate limit, and proceeds; the

backend constructs the generation prompt with the topic specification and the two-shot example; the backend calls the configured LLM provider client with the prompt; the LLM returns a JSON string containing five question objects; the backend parses the JSON, stores each Question and Option in PostgreSQL within a transaction, and returns the question data (with isCorrect omitted) to the frontend; the frontend renders the five question cards; the learner selects answers and clicks Submit; the frontend sends POST /api/quiz/submit with {quizId, answers: [{questionId, selectedOptionId}, ...]}; the backend retrieves the stored Options with their isCorrect values, computes the score, creates a QuizAttempt record, awards XP, and returns the scored results; the frontend renders per-question correctness feedback.

Daily task completion with badge award proceeds as follows: the frontend sends POST /api/daily-task/complete with {taskId}; the backend validates the token and retrieves the Task record; the backend checks that the user's container sq-{userId} is running; the backend copies the validator script from files/validators/{taskId}.sh into the container via dockerode.putArchive(); the backend executes the script via dockerode.exec() and captures the exit code; if the exit code is zero, the backend opens a Prisma transaction; within the transaction: an Attempt record is created with wasSuccess=true; the User.xp is incremented by the difficulty-appropriate amount; the User.streak is incremented if today has not yet been counted; the User.heatmap is updated with the new task completion event; each SKILL badge for the task's topic is queried and its UserBadge.progress is incremented; if any badge's progress reaches its target, the UserBadge.unlockedAt is set and an additional XP bonus is added; the transaction is committed; the backend returns the completion result including the XP awarded and any newly unlocked badges; the frontend displays the XP award animation and the badge unlock notification.

## IV. DATABASE DESIGN

### *A. Schema Philosophy and ORM Selection*

ShellQuest's relational schema is defined using Prisma, an ORM that treats the schema definition file as the single source of truth for both database structure and the generated TypeScript client. This approach ensures that every database access in the application code is type-checked at compile time: querying a field that does not exist in the schema, passing an argument of the wrong type, or accessing a relation that has not been included in a query all produce TypeScript compile-time errors rather than runtime failures. The Prisma migration system maintains a versioned history of schema changes, enabling reproducible database setup across development, staging, and production environments.

The schema is designed against PostgreSQL 16, chosen for its mature JSON column support (used for the heatmap field and the Quiz questions array), its robust indexing capabilities, and its reliability under concurrent write workloads generated by multiple simultaneous active learners. The PostgreSQL instance runs in a Docker container in local development and is managed as an AWS RDS instance in the production deployment.

### *B. Schema Overview*

The relational schema comprises ten Prisma models organized around three conceptual domains: user profile and progress, assessment and content, and AI interaction. Table V provides an overview of each model with its primary responsibility and key fields.

**TABLE V. DATABASE SCHEMA OVERVIEW**

| Model | Domain | Key Fields |
|---|---|---|
| User | Profile | id (Clerk ID), name, email, xp, streak, heatmap (JSON) |
| Badge | Gamification | name, type, difficulty, xpReward, target |
| UserBadge | Gamification | userId, badgeId, progress, unlockedAt |
| Task | Assessment | title, description, difficulty, type, validatorPath |
| Attempt | Assessment | userId, taskId, timeTaken, wasSuccess, completedAt |
| Quiz | Assessment | topic, userId, questions (JSON), attemptCount |
| Question | Assessment | quizId, statement, options (relation) |
| Option | Assessment | questionId, text, isCorrect |
| QuizAttempt | Assessment | userId, quizId, score, answers (JSON) |
| Chat | AI | userId, title, summary, messages (JSON array) |

### *C. Model Descriptions and Relationships*

The User model is the central entity of the schema. Its primary key is the Clerk user ID, a string of the form user_XXXXXXXXX, which avoids maintaining a separate UUID and ensures consistency with the authentication system. The xp field stores cumulative experience points as an integer. The streak field stores the count of consecutive days on which the user completed at least one activity. The heatmap field is a PostgreSQL JSON column storing a map from date strings (YYYY-MM-DD) to arrays of activity event objects; this denormalized structure enables efficient retrieval of the entire year's activity data in a single query, which is the most common access pattern for dashboard rendering.

The Badge and UserBadge models implement a many-to-many relationship between badge definitions and users, with the UserBadge join record carrying the progress state. Badge records are seeded during deployment and do not change at runtime; they define the name, description, category, XP reward, and numeric target of each achievement. UserBadge records are created on first progress increment for a given (user, badge) pair and updated on each subsequent increment. When the progress field reaches the target value, the unlockedAt timestamp is set and the XP reward is credited to the user.

The Task model stores both daily and weekly challenge records. The type field (discriminated by the TaskType enumeration) distinguishes between DAILY tasks (one created per calendar day) and WEEKLY tasks (maintained as a rolling batch of five). The validatorPath field stores the file system path to the bash validation script generated by the LLM at task creation time. The difficulty field (BEGINNER, INTERMEDIATE, or ADVANCED) determines the XP award on successful completion: 20, 40, or 80 XP respectively.

The Quiz, Question, and Option models form a three-level hierarchy. A Quiz record represents a generated quiz for a specific topic and user. Its associated Question records each carry the question statement (with the zero-width space uniqueness suffix appended to prevent unique constraint violations) and a relation to four Option records. The isCorrect field on the Option model stores the ground truth that is used for server-side scoring; this field is excluded from the API response to prevent client-side cheating.

The Chat model stores AI conversation sessions. The messages field is a JSON array of objects conforming to a {role, content, timestamp}

schema validated by Zod at the service layer. The summary field stores the LLM-generated one-line synopsis produced by the debounced summarization job. The title field stores the four-to-five-word session name generated by the LLM from the first message exchange.

### *D. Enumeration Types*

Three enumeration types constrain categorical fields across the schema. BadgeType defines the four badge categories: SKILL (earned by completing tasks in a specific Linux competency area), STREAK (earned for consecutive daily engagement), COLLECTION (earned for accumulating multiple badges), and GAMEPLAY (earned for discrete performance achievements such as a perfect quiz score). Difficulty defines three challenge levels: BEGINNER, INTERMEDIATE, and ADVANCED. TaskType defines three task categories: USER (user-initiated custom tasks), DAILY (one automatically generated task per calendar day), and WEEKLY (five automatically generated tasks per rolling seven-day period). These enumerations are enforced at the database level as PostgreSQL enum types, preventing invalid categorical values from being stored through any pathway including direct database access.

### *E. Indexing and Query Optimization*

The schema includes explicit indexes on the foreign key columns most frequently used in query predicates: userId on UserBadge, Attempt, QuizAttempt, and Chat; badgeId on UserBadge; taskId on Attempt; and quizId on Question and QuizAttempt. The User.heatmap JSON column is retrieved as a complete JSON object rather than queried with JSON path predicates, which is appropriate given that the heatmap is always read in its entirety for dashboard rendering. The Task model is queried with date-range filters for daily and weekly task retrieval; a composite index on (type, createdAt) supports these queries efficiently. Quiz generation queries filter by (userId, topic, createdAt) to avoid re-showing recently generated quizzes, and a composite index on these three fields is included in the schema.

### *F. Data Integrity and Consistency*

The ShellQuest data model enforces referential integrity through Prisma's relational modeling, which generates the appropriate PostgreSQL foreign key constraints for all relations. UserBadge records cannot reference non-existent Badge or User records; Attempt records cannot reference non-existent Task or User records; and Option records cannot reference non-existent Question records. These constraints prevent orphaned records from accumulating in the database through application bugs or direct database manipulation.

Concurrent write operations present a specific consistency challenge for the XP accumulation and streak tracking logic: if two requests for the same user (for example, simultaneous task completion and quiz completion) are processed concurrently, both may read the same current XP value, compute their respective increments independently, and write conflicting results. This race condition is addressed by executing all XP-incrementing operations within Prisma transactions that use SELECT FOR UPDATE locks on the User record, ensuring that concurrent increments are serialized rather than interleaved. The badge progress update, streak increment, and XP increment operations are always co-transactional, guaranteeing atomicity of the complete state update for any single learner action.

The heatmap JSON column update presents a specific challenge for concurrent writes: two simultaneous activities on the same day must both be recorded in the same date-keyed array without losing either record. The heatmap update operation reads the current heatmap JSON, appends the new activity event to the appropriate date array, and writes the updated JSON back, all within a transaction. This read-modify-write pattern is safe within a transaction because the transaction's serializable isolation level prevents another transaction from modifying the same row between the read and the write.

### *G. Schema Evolution and Migration*

The Prisma migration system maintains a versioned migration history in a dedicated _prisma_migrations table. Each migration is a SQL file generated by Prisma from the diff between the current schema and the previous migration. Migrations are applied automatically at application startup in development and applied manually during production deployments using the prisma migrate deploy command. This explicit migration management ensures that all deployment environments maintain consistent database schemas and that schema changes are tracked in version control alongside the application code changes that require them.

A migration safety convention is enforced by the team: migrations that add new columns always specify DEFAULT values or allow NULL, ensuring that the migration can complete without locking the entire table. Migrations that rename or remove columns are implemented in two stages: the first migration adds the new column (or marks the old one deprecated), the second migration (deployed after all application instances have been updated to not use the old column) removes the deprecated column. This two-phase approach prevents the downtime that would result from deploying a code change that references a column that does not yet exist in the database, or a migration that removes a column that the currently running application still reads.

### *H. Full-Text Search and Activity Querying*

The User.heatmap JSON column stores the complete activity history as a nested JSON structure keyed by date. While this denormalized structure enables efficient full-year retrieval in a single query, it does not support efficient querying of specific activity properties across the entire history—for example, "find all quiz completions with a perfect score across all dates." The current implementation performs these cross-date queries in application code by iterating over the deserialized heatmap object. For typical heatmap sizes (365 entries, each containing fewer than 10 activity events), this in-process iteration completes in under one millisecond, making the denormalized structure appropriate for the current scale.

A future evolution of the schema might introduce a separate NormalizedActivity table that stores each activity event as a relational record with proper indexed columns for userId, date, type, and type-specific fields. This normalized structure would support efficient aggregate queries such as "total quiz completions per topic" and "peak activity day of week" that are required for the planned instructor analytics dashboard. The migration from the denormalized heatmap to a normalized activity table would be implemented as a background job that reads each user's heatmap JSON and writes normalized records to the new table, followed by a cutover that switches new writes to the normalized table while maintaining backward compatibility for the existing heatmap-based dashboard.

## V. IMPLEMENTATION

### *A. Interactive Browser Terminal*

The interactive terminal is the most technically complex feature of the ShellQuest platform, requiring the coordination of four distinct technology layers: xterm.js in the browser, Socket.IO for transport, node-pty on the server, and Docker for container management. The complexity arises from the need to bridge the gap between the browser's text-based event model and the Unix pseudo-terminal interface expected by a shell process.

When the Terminal component mounts, it initializes an xterm.js Terminal instance with configuration options that match typical Linux terminal defaults: a dark background theme, a 80-column initial width, and the VT100 terminal type reported to the container. The FitAddon is attached to automatically resize the terminal dimensions to match the containing div element on window resize events. A Socket.IO

connection is established to the /terminal namespace with the Clerk JWT included in the connection query string.

On the server side, the Socket.IO connection event handler first invokes the verifyClerkAuth middleware to validate the JWT. If validation succeeds, the handler proceeds to container management. The Docker management layer queries the Docker Engine for a container named sq-{userId}; if no running container is found, it executes a container creation and start sequence using the dockerode library. The shellquest-player image, built from a Dockerfile.player at application startup if not already present, provides the Debian bookworm-slim base environment with the pre-installed utility set.

Once the container is confirmed running, node-pty is invoked with the command docker exec -it sq-{userId} /bin/bash and a set of environment variables including TERM=xterm-256color. The resulting PTY process exposes an onData callback for outgoing terminal output and a write() method for incoming keystrokes. The PTY data handler forwards all output to the connected Socket.IO client as terminal:output events. Incoming terminal:input Socket.IO events carry raw keystroke bytes that are passed directly to the PTY write() method. This bidirectional passthrough is transparent to both the container bash process, which believes it is communicating with a genuine terminal, and the xterm.js instance in the browser, which believes it is receiving output from a local process.

Terminal resize events present an additional coordination requirement. When the browser window is resized, the FitAddon recalculates the appropriate column and row counts and emits a terminal:resize event carrying the new dimensions. The server handles this event by calling PTY.resize(cols, rows), which propagates the new dimensions to the container process via a SIGWINCH signal, causing command-line programs that are sensitive to terminal dimensions (such as vim, htop, and text editors) to reflow their output correctly.

Container cleanup is managed by a debounced disconnect handler. When the Socket.IO connection is closed (either by the user navigating away or by a network interruption), a ten-minute countdown begins before the container is destroyed. If the user reconnects within this window, the countdown is cancelled and the existing container is reused, providing a seamless re-entry experience. If the countdown expires, the container is stopped and removed via the Docker API. Users can also trigger immediate container destruction and recreation through a restart button in the Terminal UI, which is useful for recovering from states where the shell has become unresponsive.

### *B. AI Chat Teaching Assistant*

The AI Chat module provides a conversational interface to an LLM-based teaching assistant specialized in Linux command-line instruction. The module is designed around three principles: conciseness (responses are limited to 120 words to maintain focus and avoid overwhelming learners with information), persistence (all conversations are stored in PostgreSQL and can be reviewed after the session), and contextual awareness (the full conversation history is included in each LLM request to enable multi-turn dialogue).

Each chat session is represented by a Chat database record with a userId, a title, a summary, and a messages JSON array. The messages array accumulates all exchanges in {role: "user"|"assistant", content: string, timestamp: ISO8601} format. When a new message is submitted, the backend retrieves the full Chat record, constructs the LLM request by prepending the system prompt to the messages array, and calls the configured LLM provider. The system prompt instructs the model to act as a friendly Linux terminal teaching assistant, to keep responses under 120 words, to avoid markdown formatting (since the chat UI renders plain text), and to focus on practical, executable explanations.

Session titles are generated by sending the first user message to the LLM with a separate prompt requesting a four-to-five-word session title. This generation step occurs only once per session, on the first message submission. The title is stored in the Chat record and displayed in the session list to help learners identify and re-open previous conversations.

The debounced summarization mechanism runs in a background timeout that is reset on every new message submission. After ten seconds of inactivity in a session (meaning no new messages have been submitted during that window), the timeout fires and sends the full conversation history to the LLM with a prompt requesting a one-line session summary. The summary is stored in the Chat.summary field and displayed in the session list as a preview. If the user submits a new message before the ten-second window expires, the timeout is cleared and rescheduled, preventing redundant summarization API calls during active conversations.

The frontend AI Chat component renders conversation history as a scrollable message list with distinct visual treatment for user and assistant messages. The user's message bubbles are right-aligned; the assistant's responses are left-aligned with a distinctive background color. An input field at the bottom of the panel handles message submission via both the Enter key and a submit button. The component scrolls to the bottom of the message list after each new message is appended, keeping the most recent exchange visible without requiring manual scrolling.

### *C. AI-Generated Adaptive Quizzes*

The adaptive quiz system generates on-demand multiple-choice assessments for five topic domains: file permissions, process management, networking basics, shell scripting, and file system navigation. These domains are selected to cover the core competency areas most frequently assessed in Linux certification examinations and most practically relevant to the use cases of the ShellQuest target learner population.

When a learner requests a quiz for a specific topic, the backend sends a generation prompt to the configured LLM requesting five multiple-choice questions, each with four answer choices and a designated correct answer, formatted as a JSON array. The prompt specifies that questions should be practical rather than definitional, that distractors should represent plausible mistakes rather than obviously incorrect answers, and that difficulty should range from beginner to intermediate to maintain accessibility while providing challenge. The LLM response is parsed as JSON; if parsing fails (due to malformed output or network errors), the system falls back to hardcoded question banks maintained for the file permissions and process management topics.

Each generated question is stored in the PostgreSQL database as a Question record with an associated set of four Option records. The isCorrect field on the correct Option is set server-side and is never included in the API response sent to the client. The zero-width space uniqueness mechanism appends an invisible Unicode character (U+200B) followed by a random four-digit suffix to each question statement before storage, preventing the database unique constraint on the statement field from being violated by near-duplicate questions generated across multiple quiz sessions.

Quiz submission is handled by a dedicated endpoint that receives the learner's selected option IDs for each question, retrieves the correct answers from the database, and computes both total score and per-question correctness. Partial credit support awards one point per correct answer, so a learner who answers three of five questions correctly receives a score of 3/5. The submission response includes per-question correctness feedback, enabling learners to identify specific knowledge gaps. XP is awarded proportionally to the score

achieved, incentivizing genuine engagement with the quiz content rather than random selection.

### *D. Daily and Weekly Task Challenges*

The task challenge system provides structured practical exercises that require learners to execute commands in their sandboxed terminal environment and submit their work for automated validation. The system operates on two time scales: one daily task that resets at midnight, and a rolling pool of five weekly tasks that is refreshed when the existing batch exceeds seven days of age.

Task generation is triggered by a check at the beginning of each day (implemented as a condition evaluated on the first API request after midnight). If no daily task exists for the current calendar date, the backend constructs a generation prompt that specifies the task topic (selected from a rotation across the five curriculum domains), the difficulty level (varying by day of week to maintain a consistent challenge curve), and a requirement that the output include a task title, a description in natural language, the expected Unix exit code convention for success, and a complete bash validation script. The LLM returns this content as a structured JSON object. The validation script is saved to the file system at files/validators/{taskId}.sh; the title, description, and difficulty are stored in the Task database record.

When a learner submits a task completion request, the backend copies the validation script from the file system into the learner's running Docker container using docker cp, then executes it inside the container with docker exec -i sq-{userId} bash < /tmp/validator.sh. The bash process receives the validation script on standard input and executes it within the container's file system and process context. An exit code of zero is interpreted as successful completion; any non-zero exit code indicates failure. The validator scripts generated by the LLM are designed to check for specific file system states, running processes, or command execution artifacts that can only be produced by genuinely completing the task.

Successful task completions create an Attempt record in the database, award XP according to the difficulty tier (20 XP for BEGINNER, 40 XP for INTERMEDIATE, 80 XP for ADVANCED), update the learner's streak counter if they have not already completed an activity on the current calendar day, and check all SKILL and STREAK badge progress thresholds for potential unlocks. This cascade of side effects is executed within a database transaction to ensure consistency: if any step fails, the entire completion is rolled back and the learner is notified of the failure without their state being partially updated.

### *E. Gamification and Badge System*

The gamification system is designed around four motivational mechanisms: experience points as a continuous progress metric, streaks as a loss-aversion retention mechanism, badges as multi-dimensional achievement recognition, and the activity heatmap as a longitudinal engagement visualization. These mechanisms are implemented to complement each other rather than operate independently; for example, maintaining a streak also contributes to STREAK badge progress, and completing tasks contributes to both XP and SKILL badge progress.

Experience points are the foundational currency of the gamification system. Every meaningful learner action awards XP: quiz completions award between 5 and 25 XP depending on score; task completions award between 20 and 80 XP depending on difficulty; badge unlocks award a badge-specific XP bonus; and AI chat sessions award a small fixed XP reward for engagement. Total XP is stored in the User.xp field and is used to compute the learner's current level via a formula that increases the XP required per level as levels advance, creating a classic RPG-style progression curve.

The streak counter is incremented once per calendar day when the learner completes any activity that awards XP. The streak is reset to zero if a day passes without any qualifying activity. The frontend displays the current streak count prominently in the dashboard header, and the heatmap provides a visual record of which days contributed to the streak. STREAK badges are unlocked at streak thresholds of 3, 7, 14, 30, and 100 consecutive days.

The badge system defines achievements across four categories. SKILL badges are associated with specific Linux competency areas (file permissions, process management, networking, scripting, file system) and unlock when the learner completes a threshold number of tasks tagged to that skill area. STREAK badges unlock at streak milestones. COLLECTION badges unlock when the learner accumulates a threshold number of total badges, providing a meta-achievement layer. GAMEPLAY badges unlock for discrete performance achievements: achieving a perfect quiz score, completing a task within a specified time limit, completing the first task, and achieving the first streak day. Each badge definition includes a name, description, SVG icon identifier, XP reward, and numeric target stored in the Badge table; the badge system is fully data-driven and new badges can be added by inserting new Badge records without any code changes.

### *F. Progress Dashboard*

The progress dashboard aggregates the learner's XP, level, streak, badge collection, topic-specific skill progress, and 365-day activity heatmap into a single view that provides both immediate feedback and longitudinal perspective on their learning journey.

The dashboard header displays the learner's current level (derived from total XP via the progression formula), XP within the current level (with a progress bar toward the next level), current streak count, and total badge count. These metrics are computed server-side from the User record and returned by the /api/progress endpoint, which also returns the badge collection with unlock states and the heatmap data.

The skill progress section displays one progress bar per curriculum topic, with the bar length determined by the number of tasks completed in that topic area relative to a fixed mastery threshold. This per-topic breakdown enables learners to identify skill areas where they are underinvested and direct their practice accordingly.

The 365-day activity heatmap renders as a grid of colored cells, one per day, spanning the 52 weeks of the trailing year. Cell color intensity encodes the number of activities completed on that day using a five-level scale from zero (empty) to four or more activities (darkest shade). Each cell is clickable; clicking a cell reveals a tooltip displaying the activities completed on that day—quiz titles, task names, badge unlocks, and chat sessions—drawn from the User.heatmap JSON column. The heatmap is rendered using a custom React component built on date-fns for date arithmetic and Tailwind utility classes for styling.

### *G. Frontend Architecture*

The React frontend is structured as a five-view application with a persistent navigation sidebar. The root App component initializes the Clerk authentication provider and the React Router DOM router. Protected routes (all views except the landing page and authentication callbacks) are wrapped by a ClerkLoaded guard that defers rendering until the authentication state has been resolved, preventing authenticated content from being displayed before the JWT is available.

Each of the five primary views (Terminal, Chat, Quiz, Tasks, Dashboard) is implemented as a top-level React component with its own data fetching logic using Axios. Data fetching is triggered by useEffect hooks that fire on component mount and, for time-sensitive data such as the daily task, on a periodic interval. LLM generation

operations (quiz generation, task completion) display loading indicators during the API call to communicate that an asynchronous operation is in progress.

The Clerk authentication UI is integrated via the SignIn and SignUp components provided by the @clerk/clerk-react package. These components handle the complete authentication flow including email verification, OAuth redirects, and session token refresh without any custom implementation required in the ShellQuest codebase. The useAuth hook provides the getToken() method used to retrieve the JWT for API calls.

### *H. Backend Service Architecture*

The Express backend is organized into route modules, middleware, service classes, and utility functions. Route modules define the HTTP method and path for each endpoint and delegate to service class methods for business logic. Middleware provides cross-cutting concerns including authentication (verifyClerkAuth), CORS configuration, and JSON body parsing. Service classes encapsulate database interactions via the Prisma client and external API interactions via the LLM connector. Utility functions provide shared logic for XP computation, date arithmetic, and badge progress evaluation.

The TypeScript compiler is configured with strict mode enabled, enforcing null checking, strict property initialization, and exact optional property types. This configuration eliminates entire categories of runtime errors at compile time. The Prisma-generated client provides complete type definitions for all database models and query results, ensuring that database interactions in service classes are type-checked against the current schema. Zod schemas are defined for all incoming request payloads and LLM API responses, providing runtime validation that catches malformed inputs before they reach business logic.

### *I. Error Handling and User Experience in the Terminal*

The terminal subsystem implements several error handling behaviors designed to maintain a smooth user experience under failure conditions. When the Docker container fails to start—for example, because the shellquest-player image is not yet available and the build process has not completed—the backend emits a terminal:error Socket.IO event carrying a user-friendly error message. The frontend displays this message inside the xterm.js terminal frame with a distinctive red color, providing contextual feedback without requiring the user to consult a separate error notification UI.

When a running container exits unexpectedly—for example, because the user executed the exit command, causing the bash process to terminate, which causes the PTY process to emit a close event—the backend emits a terminal:closed event. The frontend responds by displaying a "Terminal session ended. Click Restart to begin a new session." message and showing the Restart button prominently. This explicit state transition handling prevents the ambiguous state where the terminal appears frozen but has actually exited normally.

The PTY process may also exit due to the container being stopped by the automatic cleanup mechanism. In this case, the cleanup trigger fires the same container destruction and terminal:closed event sequence as a normal container exit. The user experience is identical: the terminal displays the session-ended message and the Restart button. The ten-minute idle timeout is calibrated to be long enough that it does not interrupt an active learner who has paused to think, but short enough that idle containers do not accumulate indefinitely on a server with limited resources.

### *J. Frontend State Management and Routing*

The ShellQuest frontend uses React's built-in useState and useEffect hooks for local component state management, supplemented by React Context for authentication state provided by Clerk. The decision to avoid a global state management library such as Redux or Zustand reflects the architectural observation that ShellQuest's five primary views are largely independent: the terminal, chat, quiz, tasks, and dashboard views do not share significant amounts of state across components, making the coordination overhead of a global state store unnecessary for the current feature set.

React Router DOM's createBrowserRouter API is used to define the route tree, which includes a root layout component that renders the navigation sidebar and an Outlet for the currently active view. The navigation sidebar uses React Router's NavLink component to apply active-state styling to the currently selected route item. Protected routes are wrapped in a ClerkLoaded component that renders a loading spinner until the authentication state has been resolved, preventing flash of unauthenticated content that would occur if protected views were rendered before the JWT validation completes.

The Clerk useAuth hook provides the getToken() method that returns the current JWT asynchronously. The Axios instance is configured with a request interceptor that calls getToken() before each request and attaches the result as the Authorization header, ensuring that all API calls carry a valid, non-expired token without requiring each component to manage token retrieval independently. The Clerk SDK automatically handles JWT rotation when tokens approach expiry, transparently issuing new tokens before the old ones expire.

Each view component implements a data loading pattern using useEffect with appropriate dependency arrays. The terminal view initializes the Socket.IO connection in a useEffect that runs once on mount and returns a cleanup function that disconnects the socket on unmount. The dashboard view loads progress data on mount and refreshes it on a 30-second interval using setInterval within a useEffect. The quiz view loads quiz history on mount and generates a new quiz in response to a user button press, tracking the loading state with a useState boolean to manage the loading spinner visibility.

### *K. Accessibility and Internationalization*

ShellQuest implements baseline accessibility features appropriate for a technical learning platform. All interactive UI elements have associated aria-label attributes where the visual context alone does not provide sufficient description for screen readers. The navigation sidebar uses ARIA landmark roles to identify it as the primary navigation region. The terminal component presents a specific accessibility challenge: xterm.js renders terminal output as canvas elements that are not intrinsically accessible to screen readers. A hidden ARIA live region is maintained that receives a text copy of recent terminal output, enabling screen reader users to hear terminal responses without requiring full terminal accessibility, which is beyond the current implementation scope.

The platform is currently implemented and documented in English only. Internationalization (i18n) is identified as a future development priority, particularly given the platform's target audience in Indian educational institutions where English is widely used as the language of technical instruction but not always the preferred natural language for learning materials. The codebase is structured to support i18n through string externalization: no user-visible strings are hardcoded in JSX templates; all interface labels, error messages, and instructional text are defined in a central strings module that would be replaceable by language-specific modules in a future i18n implementation.

The xterm.js terminal emulator supports a rich set of ANSI escape sequence codes that cover cursor movement, text color and styling, and terminal control operations. The shellquest-player Docker image is configured with TERM=xterm-256color to ensure that shell programs receive accurate capability information and produce appropriately formatted output. This is particularly important for interactive

programs such as vim, which rely on cursor positioning escape sequences to render their user interface correctly. Without accurate TERM configuration, vim and similar programs produce malformed output that is difficult for learners to interpret.

The terminal resize handling involves a subtle coordination between three components: the browser window, the xterm.js viewport, and the PTY process inside the container. When the browser window is resized, the ResizeObserver attached to the terminal container div fires, causing the FitAddon to recalculate the optimal column and row count based on the current div dimensions and the configured character cell size. The FitAddon calls terminal.resize(cols, rows) to update the xterm.js rendering, and simultaneously emits a terminal:resize Socket.IO event carrying the new dimensions. The server receives this event and calls pty.resize(cols, rows), which sends a SIGWINCH signal to the bash process and all its children, causing them to query the new terminal dimensions and reflow their output. This three-way synchronization ensures that the displayed terminal and the running process always agree on the terminal dimensions.

The scrollback buffer in xterm.js is configured to retain 10,000 lines of history, enabling learners to scroll back and review the output of commands executed earlier in their session without losing context. This buffer is maintained in JavaScript memory within the browser; it does not persist across page refreshes or session reconnections. When a learner reconnects to an existing container after a temporary disconnection, the PTY resumes from its current state but the xterm.js buffer is cleared, so the learner sees a fresh terminal view rather than the previous session output. This behavior is a limitation of the current implementation; persistent scrollback across reconnections would require server-side buffering of PTY output, which is deferred to a future development iteration.

Terminal input handling includes special processing for paste events. The xterm.js onPaste handler is configured to strip control characters from pasted text that could cause unintended effects when injected into the PTY stream. Specifically, the BEL character (U+0007) and other C0 control characters except TAB, LF, and CR are stripped from paste content before forwarding to the Socket.IO terminal:input event. This prevents a class of terminal injection attacks where a malicious web page could trick the user into pasting content that executes commands in their terminal without their knowledge.

The container naming scheme sq-{userId} has an important implication for the multi-tenancy model: a single physical server can host arbitrary numbers of users without namespace collisions, bounded only by available memory and CPU. In practice, each idle container consumes approximately 8-12 MB of memory (the Debian bookworm-slim base with no active processes), while an active container with a running bash session and a few utility programs consumes approximately 20-30 MB. On an EC2 t3.medium instance with 4 GB of RAM, this supports approximately 100-130 concurrent container instances with comfortable headroom for the backend process and operating system overhead.

The quiz topic selection UI presents the five available domains as a set of five buttons arranged in a topic picker panel. Each button displays the topic name and an icon representing the topic domain: a lock icon for file permissions, a gear icon for process management, a network icon for networking basics, a script icon for shell scripting, and a folder icon for file system navigation. When a topic is selected, the Generate Quiz button becomes active and a brief description of the topic scope is displayed in a tooltip, helping learners understand what knowledge will be assessed before they commit to generating a quiz.

Quiz question display uses a numbered list layout with the question statement at the top of each question card, followed by four answer choice buttons labeled A through D. Selected answers are highlighted with a distinctive border color; the selection state is maintained in React component state and cleared when a new quiz is generated. After submission, the card for each question updates to show either a green checkmark (correct answer) or a red X (incorrect answer) alongside the learner's selected choice, with the correct answer highlighted in green regardless of whether the learner selected it. This immediate, specific feedback enables learners to identify exactly which questions they answered incorrectly and what the correct answer was, supporting self-directed remediation.

The partial credit scoring model awards one point per correctly answered question out of five possible points. The XP award for a quiz attempt is calculated as (score / 5) × 25 XP, rounded to the nearest integer, so a perfect score of 5/5 awards 25 XP, a score of 4/5 awards 20 XP, and a score of 0/5 awards 0 XP. This proportional model incentivizes genuine engagement with every question rather than the risk-minimizing strategy of random selection, which on a five-question quiz produces an expected score of 1/5 and thus only 5 XP on average. XP is awarded only once per quiz attempt; learners cannot earn additional XP by retaking the same generated quiz.

The LLM prompt for quiz generation includes several specific constraints that improve question quality. Questions must require knowledge application rather than mere recall: "Which command lists all running processes with memory usage?" (application) rather than "What does the ps command do?" (recall). Distractors must be plausible commands or options that a learner might reasonably confuse with the correct answer, rather than obviously incorrect options that make the quiz trivially easy. Each question must have exactly one clearly correct answer, not two potentially correct answers that could be distinguished only through fine-grained interpretation. These constraints are specified in natural language in the system prompt and produce markedly better question quality than an unconstrained generation prompt.

The fallback question bank for file permissions contains ten questions covering chmod, chown, the permission bit notation (rwxrwxrwx), the numeric octal notation (chmod 755, chmod 644), and special permission bits (setuid, setgid, sticky bit). The fallback question bank for process management contains ten questions covering ps, top, kill, nice, renice, pgrep, pkill, and job control commands (bg, fg, jobs). These hardcoded questions are rotated randomly when the fallback is invoked, so learners are unlikely to see the same question repeatedly even across multiple fallback-triggered quiz sessions. The fallback banks cover the most frequently assessed topics in Linux certification examinations, ensuring that the fallback experience is educationally valuable rather than merely a placeholder.

The task generation prompt structure is carefully designed to ensure that the generated output can be reliably parsed into its required components. The prompt specifies a JSON output format with four required fields: title (a concise task name of five to ten words), description (a natural-language explanation of what the learner must accomplish, between fifty and one hundred words), difficulty (one of the three enumeration values: BEGINNER, INTERMEDIATE, or ADVANCED), and validator (a complete bash script that tests whether the task has been correctly completed). The prompt explicitly states that the output must be a valid JSON object and nothing else, with no surrounding prose or markdown code fences. Despite this specification, the LLM occasionally produces output with markdown code fences or leading explanatory text; the parser trims leading and trailing whitespace and strips code fences before the JSON.parse() call.

Daily task difficulty is varied according to a weekly schedule to provide a structured challenge curve. Monday and Tuesday tasks are generated at BEGINNER difficulty, suitable for learners who are new to the topic or who are using the platform for the first time. Wednesday

and Thursday tasks are generated at INTERMEDIATE difficulty, requiring knowledge of command options and composition techniques. Friday tasks are generated at ADVANCED difficulty, requiring multi-step workflows or knowledge of less common commands. Weekend tasks alternate between BEGINNER and INTERMEDIATE, with a small XP bonus to incentivize weekend engagement. This schedule is implemented as a lookup table indexed by the ISO day-of-week number of the current date.

The weekly task batch provides five tasks simultaneously, allowing learners to choose the task that best matches their current knowledge and available time. The batch always includes at least one task of each difficulty level, ensuring that learners of all experience levels find an appropriate challenge. The remaining two tasks in the batch are selected by the LLM based on the rotation schedule across the five topic domains. The batch is refreshed when the oldest task in the current batch is more than seven days old, providing a rolling window of challenges that prevents learners from encountering tasks they have already seen.

The bash validation script execution mechanism deserves detailed attention because it represents a boundary between the trusted server environment and the untrusted learner environment. The validator script is generated by the LLM and stored on the server file system, but it executes inside the learner's Docker container. This execution model has two important security properties: first, the script cannot access or modify any files outside the container's file system namespace; second, the script runs with the same user permissions as the learner's bash shell, so it cannot perform privileged operations that the learner could not perform directly. The validator script is treated as potentially containing arbitrary bash code (because it was generated by an LLM that might produce unexpected constructs), but its blast radius is bounded to the learner's own container, which can be destroyed and recreated without any lasting effect.

The time measurement for task completion records the elapsed time between the task first being displayed to the learner (when they load the task list) and the moment of successful validation. This time is stored in the Attempt.timeTaken field in seconds. The GAMEPLAY badge for "Speed Demon" is awarded for completing an ADVANCED task in under five minutes, incentivizing learners who are competent enough to complete the task quickly. The time measurement is imprecise in the sense that it measures wall clock time including any time the learner spent reading the task, browsing documentation, or being distracted; it is not a measure of purely on-task execution time. This imprecision is acceptable because the badge threshold is set conservatively enough (five minutes for an advanced task) that it filters out purely casual completion attempts.

The XP-to-level formula follows an exponential progression curve designed to provide rapid early level advancement (to reward new learners immediately) while making later levels genuinely challenging to achieve. The formula is level = floor(sqrt(totalXP / 100)), which means that Level 1 requires 100 XP, Level 2 requires 400 XP, Level 3 requires 900 XP, Level 4 requires 1600 XP, and Level 10 requires 10,000 XP. This formula is applied to the total cumulative XP, so the XP required to advance from one level to the next increases with each level. A learner who completes one INTERMEDIATE task (40 XP), one BEGINNER quiz (5 XP), and one AI chat session (10 XP) per day will accumulate approximately 55 XP per day, reaching Level 1 after two days and Level 2 after about a week of consistent use.

The streak calculation runs daily at midnight server time using a Node.js cron job. The cron job queries all users who have at least one activity in the previous 24-hour window and verifies that their streak counter is consistent with the presence or absence of activity. If a user had activity yesterday but not on any prior day, their streak is set to 1. If a user had activity yesterday and had a non-zero streak on the day before that, their streak is incremented by 1. If a user had no activity yesterday, their streak is reset to 0. This calculation is idempotent: running it multiple times in a day has no additional effect because it queries the most recent activity timestamp rather than incrementing a counter without a guard condition.

Badge progress updates are triggered as side effects of successful task completions, quiz submissions, badge unlocks (for COLLECTION badges), and chat session initializations. The badge progress evaluation logic queries all Badge records and for each badge, determines the current UserBadge progress value and checks whether it has reached the target. This evaluation is performed as a batch operation after each triggering event, rather than incrementally during the event handler, to ensure that COLLECTION badges (which depend on the count of other unlocked badges) are evaluated correctly after all other badge updates have completed. Badge XP rewards are credited to the user in the same transaction as the badge unlock, ensuring atomicity.

The activity heatmap rendering algorithm in the React frontend generates a 52-week grid of cells, each representing one day. The grid is oriented with weeks as columns and days as rows, so that the most recent week appears on the right side of the grid and the oldest week on the left. Days in the future are rendered as empty cells with a distinct visual treatment to distinguish them from days with no recorded activity. The cell intensity is mapped to the activity count for that day using a five-level color scale from near-white (zero activities) to deep green (four or more activities), using CSS custom properties that respect the user's preferred color scheme via the prefers-color-scheme media query. Clicking a cell dispatches a React state update that renders a tooltip showing the date, total activity count, and a bulleted list of activity titles for that day.

The heatmap data structure stores activities as a JSON object keyed by date string (format YYYY-MM-DD). Each date key maps to an array of activity event objects conforming to the Zod discriminated union schema. On heatmap reads, the frontend iterates over the 365-day date range, looking up each date in the heatmap object and rendering the cell intensity based on the array length at that key. Dates not present in the heatmap object (days with no activity) render as intensity level zero. This sparse storage approach is efficient for typical usage patterns where most days have either zero or a small number of activities, avoiding the overhead of storing explicit zero values for the hundreds of inactive days in a typical learner's 365-day history.

### *L. Terminal Session Lifecycle Management*

The complete lifecycle of a terminal session encompasses six distinct phases: authentication, container provisioning, PTY attachment, active session, idle monitoring, and cleanup. Each phase has clearly defined entry conditions, exit conditions, and failure modes. Understanding this lifecycle is essential for diagnosing session-related issues and designing extensions to the terminal subsystem.

The authentication phase begins when a Socket.IO connection request arrives at the /terminal namespace. The connection handler extracts the auth.token field from the handshake query string and passes it to the verifyClerkAuth function. This function calls Clerk's SDK verifyToken() method, which performs asymmetric signature verification using Clerk's published RSA public key (cached in the Clerk SDK after the first retrieval). If verification succeeds, the decoded user ID is extracted and the connection proceeds. If verification fails (invalid signature, expired token, or malformed token), the socket is immediately disconnected with a disconnect reason of "authentication_error". This failure is logged with the remote IP address and the failure reason for security monitoring purposes.

The container provisioning phase begins immediately after successful authentication. The Docker management layer queries the Docker Engine for containers matching the name filter sq-{userId}. The query returns all containers with that name regardless of their current state; the handler specifically checks for containers in the "running" state. If a running container exists, provisioning is skipped and the handler proceeds directly to PTY attachment. If no running container exists but a stopped container with the correct name exists, the handler starts the stopped container and waits for it to transition to the running state. If no container with the correct name exists, the handler creates a new container from the shellquest-player image, starts it, and waits for the running state.

Container creation uses a set of fixed configuration options specified as a dockerode ContainerCreateOptions object: the image is shellquest-player; the name is sq-{userId}; the Tty and OpenStdin flags are set to true to enable PTY-compatible behavior; the HostConfig includes a memory limit of 268,435,456 bytes (256 MB), a CPU quota of 50,000 microseconds per 100,000-microsecond period (0.5 CPU), and an auto-remove flag set to false (to allow explicit cleanup). A network mode of "host" is used in the local development environment to simplify connectivity; in the production EC2 deployment, containers are assigned to a dedicated bridge network.

PTY attachment creates a node-pty PTY instance using pty.spawn() with "docker" as the command and ["exec", "-it", "sq-{userId}", "/bin/bash"] as the arguments. The cols and rows parameters are set from the current terminal dimensions provided in the Socket.IO handshake options. The onData callback is registered to forward PTY output to the Socket.IO client. The Socket.IO "terminal:input" event handler is registered to call pty.write() with the received keystroke bytes. The PTY is stored in a Map keyed by the socket ID for retrieval in subsequent event handlers.

The idle monitoring phase begins after PTY attachment. A Node.js timer is set for 600,000 milliseconds (ten minutes). This timer is reset whenever a "terminal:input" Socket.IO event is received, indicating ongoing user activity. When the timer fires, the idle cleanup procedure runs: the PTY is killed, the Docker container is stopped and removed, and the Socket.IO socket is emitted a "terminal:closed" event. If the socket is still connected when cleanup runs, the client displays the session-ended message and the Restart button; if the socket has already disconnected (due to navigation or network interruption), the cleanup proceeds silently.

### *M. Frontend Component Architecture*

The frontend application is organized as a hierarchy of React components following a feature-based directory structure. The top level of the src directory contains the App component (routing and global providers), a components directory (shared UI components), a pages directory (top-level view components), a hooks directory (custom React hooks), a services directory (Axios API client functions), and a utils directory (shared utility functions).

The Terminal page component is the most complex view component in the application. It manages the following state: the Socket.IO connection reference, the xterm.js Terminal instance reference, the terminal:ready status boolean (used to determine whether the terminal overlay message should be shown), the container provisioning state (loading, ready, or error), and the restart countdown timer. The component renders a div element that serves as the xterm.js mount target, a transparent overlay that shows a loading spinner during container provisioning, and a controls toolbar containing the Restart button and a terminal settings panel for font size adjustment.

The AI Chat page component manages a two-panel layout: a session list panel on the left displaying all of the user's chat sessions with their titles and summaries, and a conversation panel on the right displaying the messages of the currently selected session. The component maintains state for the session list data, the currently selected session ID, the messages of the selected session, the input field value, and the loading state of the current message submission. A useEffect hook loads the session list on mount; a second useEffect hook loads the messages for the selected session when the selectedSessionId state changes.

The Quiz page component presents a multi-step UI flow managed through a step state variable: "select_topic" (showing the topic picker), "generating" (showing a loading spinner), "taking_quiz" (showing the question cards), and "results" (showing the score and per-question feedback). The component maintains state for the selected topic, the generated quiz data (received from the API), the learner's answer selections (stored as a Map from question ID to selected option ID), the submission result (received from the API), and the loading state. The quiz flow is linear and does not allow backward navigation once questions have been answered, enforcing a genuine test-taking experience.

The Tasks page component presents two sections: the daily task card (showing today's task title, description, difficulty badge, and a Complete button) and the weekly tasks list (showing five task cards in a grid). Each task card includes a status indicator that shows whether the task has already been completed by the current user (stored in the Attempt table). Completed tasks display a green checkmark and the time taken; incomplete tasks display the Complete button. The completion submission triggers a loading state overlay on the task card, preventing double-submission and providing visual feedback that the server-side validation is in progress.

The Dashboard page component is the most data-intensive view, rendering the user's XP level indicator, streak counter, badge grid, skill progress bars, and the 365-day heatmap. The badge grid renders each Badge record as a circular icon with a progress bar showing completion percentage; unlocked badges are displayed with full color and a gold border, while locked badges are displayed in grayscale. The skill progress bars use a custom SkillBar component that accepts a topic name, current task count, and mastery threshold as props and renders a labeled horizontal bar. The heatmap is rendered by a dedicated HeatmapGrid component that manages the cell click state for tooltip display independently of the parent page component.

### *N. Backend Middleware Chain*

The Express request pipeline for all protected API routes passes through a sequence of middleware layers before reaching the route handler function. Understanding this middleware chain is important for debugging request failures and for designing new endpoints that must interact correctly with the authentication and validation infrastructure.

The first middleware layer is the cors middleware, which sets the Access-Control-Allow-Origin response header to the configured FRONTEND_URL environment variable and the Access-Control-Allow-Headers header to include the Authorization header required for JWT delivery. OPTIONS preflight requests are handled by the cors middleware and responded to immediately with a 200 status, without proceeding to the subsequent layers.

The second layer is the express.json() middleware, which parses the request body from JSON format and attaches the parsed object to req.body. Requests with a Content-Type header other than application/json are rejected with a 415 Unsupported Media Type status by this middleware. The body size limit is configured to 1 MB, sufficient for all expected payloads while preventing memory exhaustion from unexpectedly large request bodies.

The third layer is the verifyClerkAuth middleware, which extracts the Bearer token from the Authorization header, validates it using the

Clerk SDK, retrieves or creates the corresponding User record from PostgreSQL, and attaches the user object to req.user. If authentication fails at any step, this middleware calls next() with an error object, which triggers the global error handler middleware to return a 401 Unauthorized response. If authentication succeeds, it calls next() without an argument, allowing the request to proceed to the rate limiting middleware.

The fourth layer is the per-route rate limiting middleware, applied only to LLM-consuming endpoints. This middleware queries a Redis cache (or the PostgreSQL database as a fallback when Redis is not available) for the number of requests made by the authenticated user within the current rate limit window. If the count is below the threshold, the count is incremented and the request proceeds. If the count is at or above the threshold, the middleware returns a 429 Too Many Requests response with a Retry-After header indicating when the rate limit window resets. Finally, the route handler function processes the request and calls res.json() to send the response.

### *O. LLM Prompt Engineering Details*

The quality of ShellQuest's AI-driven features depends critically on the prompts used to invoke the LLM. Each prompt is designed through an iterative refinement process that evaluates the model's outputs across a sample of inputs and adjusts the prompt to improve consistency, accuracy, and format compliance. This section documents the key prompting strategies employed in each AI feature.

The AI chat system prompt instructs the model to adopt the persona of a "friendly Linux terminal teaching assistant" who has the following behavioral constraints: responses must be under 120 words; technical commands must be mentioned in their exact, executable form; responses must not use markdown formatting (no bold, italic, or code fences); and when the user's question is ambiguous, the assistant should ask a single clarifying question rather than providing an answer that may not match the user's need. The 120-word limit is implemented through a post-processing step that truncates responses at word boundaries if the model produces longer output despite the instruction; this truncation is a fallback for the rare cases where the model does not comply with the length constraint.

The quiz generation prompt specifies the output format as a JSON array of five objects, each with the fields question (string), options (array of four strings labeled A through D), and answer (one of "A", "B", "C", or "D"). The prompt includes a two-shot example showing a well-formed question and a poorly-formed question (one with an obvious correct answer and transparent distractors) with an explanation of why the poor question fails the quality criteria. Two-shot prompting with a negative example significantly improved the quality of the distractors in generated questions compared to a zero-shot prompt, based on qualitative evaluation of outputs across 50 generation attempts per topic domain.

The task generation prompt uses a chain-of-thought approach, asking the model to first reason about what the learner must do to complete the task (the "chain of thought" step), then specify the exact bash validator that would confirm completion, and finally write the natural-language task description based on what the validator will check. This ordering—validator before description—was adopted after early experiments with description-first ordering revealed that the LLM sometimes generated descriptions that were ambiguous about whether directory creation, file modification, or process execution was required, leading to validators that tested a different specific behavior than the description implied.

The chat session title generation prompt requests a title of "exactly four to five words, capturing the main Linux concept discussed, without articles or filler words." The brevity constraint and the specificity requirements produce titles like "chmod file permission management" and "find command recursive search" rather than generic titles like "Linux commands question" that would not help learners identify previous sessions in the session list. Title generation uses a lower temperature setting (0.3) than the main chat response (0.7) to produce consistent, professional titles rather than creative variations.

### *P. Database Query Patterns and Performance*

The most frequently executed database queries in the ShellQuest application are the authentication upsert (executed on every protected request), the progress data assembly (executed on each dashboard page load), and the activity heatmap update (executed on each XP-awarding event). The performance characteristics of each query directly affect the perceived responsiveness of the corresponding user-facing operations.

The authentication upsert uses Prisma's upsert() operation with the Clerk user ID as the where clause key. For returning users (the vast majority of requests after initial registration), this operation resolves as an update with an empty update clause (only the updatedAt timestamp is modified), which PostgreSQL can execute as a single index lookup followed by a no-op update. The query plan shows an index scan on the User table's primary key index with an estimated cost of 0.43 for a user table with 1,000 rows, confirming that authentication overhead is negligible relative to the subsequent business logic.

The progress data assembly executes five queries: a User record fetch (by primary key, O(log n)), an aggregated XP-to-level computation (client-side, from the User record), a UserBadge join query (by userId, with badge relation eagerly loaded), a Task completion count grouped by topic (aggregate query with group by, using the (userId, taskId) composite index on Attempt), and a Quiz attempt summary (by userId, with recent-first ordering). These five queries are issued in parallel using Promise.all() rather than sequentially, reducing the total latency to the maximum of the individual query times rather than their sum. For a user with 200 task attempts and 50 quiz attempts, the most expensive query (Task completion count) takes approximately 12 milliseconds with the composite index in place.

The heatmap update query reads the current heatmap JSON column, appends a new activity event to the appropriate date array, and writes the updated JSON back. This read-modify-write pattern is executed within a transaction to prevent race conditions. The JSON manipulation is performed in JavaScript (by the Prisma service layer) rather than in PostgreSQL (using JSONB path operators) because the application layer manipulation is simpler to implement and maintain, and the transaction isolation guarantees that no concurrent modification can occur between the read and the write. For a heatmap JSON object with 365 date entries, each containing an array of activity events, the JSON parsing and serialization overhead is approximately 2 milliseconds, well within the acceptable latency budget for this operation.

### *Q. Testing Infrastructure and Practices*

The ShellQuest backend testing infrastructure uses Jest as the test runner and the @testing-library/react package for frontend component testing. The test suite is organized into three categories: unit tests for service class methods, integration tests for API endpoints, and end-to-end tests for complete user journeys.

Unit tests for service class methods use Jest's mocking capabilities to isolate the service under test from its dependencies. The Prisma client is mocked using a jest.mock() call that replaces all Prisma methods with Jest spy functions that return configurable mock responses. The LLM connector is mocked similarly. Unit tests verify that service methods call the expected Prisma operations with the expected arguments, compute the correct derived values (XP totals, level numbers, badge progress thresholds), and handle error cases correctly (LLM API failures, missing database records, malformed input).

Integration tests for API endpoints use the supertest library to make real HTTP requests to an Express app instance configured with a test database (a separate PostgreSQL database populated with seed data by a beforeAll() hook and truncated by an afterEach() hook). Integration tests verify the complete request-response cycle including middleware behavior: authentication failure responses, validation error responses, correct resource creation and retrieval, and rate limiting enforcement. The test database is separate from the development database to prevent test data from contaminating the development environment.

End-to-end tests for terminal functionality are the most challenging to automate because they require a running Docker Engine and real container management. These tests are implemented as a separate test suite tagged as "integration" that is excluded from the default CI test run and executed separately in a Docker-enabled environment. The terminal end-to-end tests verify container creation timing, PTY data forwarding, terminal resize propagation, idle cleanup execution, and container reuse across reconnections. Each test uses a unique userId suffix to prevent test container naming conflicts when multiple test cases run concurrently.

## VI. API DESIGN AND SECURITY

### *A. REST API Design Principles*

The ShellQuest REST API follows resource-oriented design principles consistent with conventional REST conventions. Resources are identified by noun-based URL paths; collection endpoints respond to GET for retrieval and POST for creation; authentication is enforced uniformly on all protected routes via the verifyClerkAuth middleware. All responses are JSON-encoded and include appropriate HTTP status codes: 200 for successful retrieval, 201 for successful creation, 400 for validation errors, 401 for authentication failures, 404 for missing resources, and 500 for server-side errors. Error responses include a structured JSON body with a message field and, where applicable, a details field providing Zod validation error information.

### *B. API Endpoint Summary*

Table VI summarizes all REST and WebSocket endpoints exposed by the ShellQuest backend.

**TABLE VI. API ENDPOINT SUMMARY**

| Method | Endpoint | Auth | Description |
|---|---|---|---|
| GET | /health | No | Server health check and version info |
| POST | /api/ai-chat | JWT | Send message, create/continue session |
| GET | /api/ai-chat | JWT | List all chat sessions for user |
| GET | /api/ai-chat/:id | JWT | Retrieve specific session with messages |
| POST | /api/quiz/generate | JWT | Generate 5 MCQs for a topic |
| POST | /api/quiz/submit | JWT | Submit answers, receive score and feedback |
| GET | /api/quiz/history | JWT | List user quiz attempt history |
| GET | /api/daily-task | JWT | Get today's and this week's tasks |
| POST | /api/daily-task/complete | JWT | Validate and record task completion |
| GET | /api/badges | JWT | Get all badges with user progress |
| GET | /api/progress | JWT | Get comprehensive dashboard data |
| POST | /api/progress/activity | JWT | Record a manual activity event |
| WS | Socket.IO / | JWT | Real-time terminal I/O stream |

### *C. Authentication Flow*

Authentication in ShellQuest follows a federated identity model in which Clerk acts as the identity provider and the ShellQuest backend acts as a relying party. The complete authentication flow proceeds as follows. A user visits the ShellQuest frontend and chooses to authenticate via email, Google OAuth, or GitHub OAuth. Clerk presents the appropriate authentication UI and, on successful authentication, issues a session JWT signed with Clerk's RSA private key. The frontend stores this JWT in browser memory (via the Clerk session context, not localStorage, to avoid XSS exposure) and retrieves it via the getToken() method for each API call.

On every protected API request, the frontend attaches the JWT as a Bearer token in the Authorization header. The server-side verifyClerkAuth middleware calls the Clerk SDK's verifyToken() function, which validates the JWT signature against Clerk's public key, checks the expiry timestamp, and returns the decoded payload including the Clerk user ID. The middleware attaches this user ID to the Express request object for use by downstream route handlers. If the token is missing, malformed, or expired, the middleware returns a 401 Unauthorized response immediately without forwarding the request to the route handler.

First-time users are automatically provisioned in the local database. When verifyClerkAuth successfully validates a token for a user ID that does not exist in the User table, the middleware executes a Prisma upsert operation that creates a new User record with default values (xp=0, streak=0, empty heatmap). This approach eliminates the need for a separate user registration endpoint and ensures that the database user record is always synchronized with the Clerk identity before any route handler accesses it.

For WebSocket connections, JWT validation follows the same logic but uses a different delivery mechanism. The Clerk JWT is included in the Socket.IO connection handshake as the auth.token query parameter. The server-side connection event handler extracts this token and passes it to the same verifyClerkAuth logic used for REST requests. If validation fails, the socket is immediately disconnected with an authentication error event.

### *D. Security Architecture*

The ShellQuest security architecture addresses threats at multiple layers. Container isolation ensures that each learner's terminal session is confined to a Docker container that shares no file system, process, or network namespace with other learners' containers. A compromised container cannot access other containers' file systems or processes by construction of Docker's namespace isolation model. The containers run as non-root users where possible (the shellquest-player image is configured to run bash as a restricted user), limiting the blast radius of any command execution within the container.

The CORS configuration restricts the Access-Control-Allow-Origin header to the configured frontend origin URL, preventing cross-origin API requests from unauthorized domains. In production, this is set to the Vercel deployment URL. The Zod validation middleware rejects any request payload that does not conform to the expected schema, preventing injection attacks through malformed input. SQL injection

is not a concern because all database interactions go through the Prisma ORM's parameterized query generation, which never interpolates user-supplied values directly into SQL strings.

The validator script execution mechanism requires particular attention. When a task validator script is copied into a learner's container and executed, it runs with the permissions of the bash process inside the container. Since the validator is generated by the LLM and stored on the server file system, a malicious learner cannot supply their own validator; they can only influence whether the validator's conditions are satisfied by the state of their container. The validator runs inside the learner's own container, so it can only observe and affect that container's state, not the host system or other containers.

Rate limiting on LLM API endpoints (quiz generation, task completion, AI chat) is enforced via a per-user counter stored in PostgreSQL. Users who exceed the configured rate limit threshold within a rolling time window receive a 429 Too Many Requests response. This prevents both inadvertent runaway client behavior and deliberate API abuse from driving excessive LLM API costs.

### *E. WebSocket Event Protocol*

The Socket.IO terminal namespace exposes a defined set of events that constitute the terminal communication protocol. Client-to-server events include terminal:input (carrying a string of keystroke bytes), terminal:resize (carrying {cols: number, rows: number}), and terminal:restart (requesting container destruction and recreation). Server-to-client events include terminal:output (carrying a string of terminal output bytes), terminal:ready (confirming that the container and PTY are initialized and accepting input), terminal:error (carrying an error message string), and terminal:closed (indicating that the PTY process has exited). This event protocol is documented in the codebase as a TypeScript interface, enabling the TypeScript compiler to verify that all event emissions and handler registrations use the correct event names and payload types.

The protocol includes a handshake sequence that ensures the frontend does not send terminal input before the container and PTY are ready. When the Socket.IO connection is established, the backend immediately begins container provisioning. It emits terminal:ready only after the PTY process is spawned and responding to input. The frontend's xterm.js instance buffers any keystrokes typed during this initialization window and flushes them to the socket after the terminal:ready event is received, preventing lost keystrokes during the two-to-three-second cold-start provisioning period.

### *F. API Versioning and Backward Compatibility*

The current API does not implement versioning, as all clients (the ShellQuest frontend) are deployed and updated simultaneously with the backend, eliminating the need to support multiple concurrent API versions. If third-party integrations or mobile applications are developed in the future, API versioning via URL path prefixes (/api/v1/, /api/v2/) will be introduced to maintain backward compatibility across independently deployed client and server versions. The structured JSON response format with consistently named fields is designed to be stable and forward-compatible: new fields can be added to responses without breaking existing clients that do not reference them.

API documentation is maintained as inline JSDoc comments on each route handler function, describing the request and response schemas, authentication requirements, and error conditions. These comments are used by the VS Code IDE to provide inline documentation for developers working with the codebase. A formal OpenAPI specification generation step using the swagger-jsdoc library is planned for the next development iteration, which will produce machine-readable API documentation that can be used to generate client SDKs, API testing suites, and human-readable API reference documentation automatically from the JSDoc comments.

The quiz generation endpoint at POST /api/quiz/generate accepts a JSON body with a topic field constrained to one of the five valid topic strings. The endpoint first checks the rate limiting counter for the requesting user to enforce the maximum of five quiz generation requests per hour. If the rate limit is not exceeded, it increments the counter and proceeds to invoke the LLM service. The quiz generation request includes a fixed random seed appended to the prompt to reduce the probability of generating the same questions as a recent session for the same topic, though the LLM's inherent non-determinism (temperature > 0) provides the primary source of variation. The response includes the quiz ID (for use in the subsequent submit request) and the five questions with their four answer options, with the isCorrect field omitted from each option.

The task completion endpoint at POST /api/daily-task/complete accepts a JSON body with a taskId field. The endpoint retrieves the Task record to verify it exists and has not expired, then retrieves the user's running container via the Docker management layer. The validator script is copied into the container using dockerode's putArchive method (which creates a tar archive in memory and decompresses it inside the container), then executed using dockerode's exec method with stdin set to the validator script content. The exec result's exit code determines success or failure. The entire operation has a configurable timeout of thirty seconds, after which the exec is terminated and the completion is marked as failed with a timeout error message returned to the client.

## VII. DOCKER AND DEPLOYMENT ARCHITECTURE

### *A. Local Development Environment*

The local development environment is orchestrated using Docker Compose across three services defined in docker-compose.yml. The first service is a PostgreSQL 16 Alpine container exposing port 5432, initialized with the POSTGRES_USER, POSTGRES_PASSWORD, and POSTGRES_DB environment variables from a shared .env file. The second service is the ShellQuest backend, built from the project's Dockerfile, exposing port 4000, configured to wait for the database service to be healthy before starting, and granted access to the host Docker socket via a volume mount (/var/run/docker.sock). The third service is an Adminer container exposing port 8080 for database inspection during development.

The Docker socket volume mount is the key enabler of the Docker-in-Docker pattern. By mounting the host's Docker socket inside the backend container, the backend can issue Docker Engine API commands that create and manage sibling containers on the host, rather than nested child containers within the backend container. This sibling container model is preferred over true DinD (which requires the backend container to run with privileged mode) because it has lower overhead, simpler networking, and better resource isolation.

### *B. Container Image Architecture*

The system uses two distinct container images. The backend image is built from a Node.js 18 Alpine base image, copies the TypeScript source, installs dependencies via npm ci --production, runs the TypeScript compiler, and sets the entry point to the compiled main.js. The shellquest-player image is built from debian:bookworm-slim and installs the learner tool set: bash, coreutils, findutils, grep, sed, gawk, nano, vim-tiny, tree, curl, netcat-openbsd, and sudo. The player image is built at application startup if not already present, using dockerode to invoke a build from Dockerfile.player. This lazy build approach ensures that the image is always up to date with the current Dockerfile without requiring it to be pre-built as part of the CI/CD pipeline.

Container resource limits are configured at creation time via the Docker Engine API. Each learner container is limited to 256 MB of memory and 0.5 CPU cores, preventing any individual learner from consuming a disproportionate share of host resources. These limits are generous enough to support the command-line tools installed in the image but restrictive enough to allow dozens of simultaneous containers on a reasonably sized EC2 instance.

### *C. Production Deployment*

The production deployment comprises two independently deployed components. The frontend is deployed to Vercel, which provides automatic deployment on git push to the main branch, CDN-served static assets, and SSL termination. The backend is deployed to an AWS EC2 instance running Amazon Linux 2. The EC2 instance hosts the Dockerized backend service, manages the PostgreSQL database as an AWS RDS instance for durability and managed backups, and exposes the backend API over HTTPS via an AWS Application Load Balancer with an ACM-managed certificate.

The separation of frontend and backend deployment provides several operational benefits. Frontend updates (UI changes, bug fixes that do not affect API contracts) can be deployed without restarting the backend, eliminating downtime for active terminal sessions. Backend updates require a service restart, which terminates active WebSocket connections and destroys running learner containers; a rolling restart strategy is planned for the production roadmap to minimize disruption.

Environment configuration is managed through separate .env files for development and production, with sensitive values (Clerk API keys, LLM API keys, database credentials) stored in AWS Secrets Manager in production and retrieved at container startup. This approach prevents sensitive credentials from being committed to the source code repository or baked into container images.

### *D. Scalability Considerations*

The current architecture supports vertical scaling through the addition of CPU and memory resources to the EC2 instance. Horizontal scaling requires addressing two stateful dependencies: the PostgreSQL database (which can be horizontally scaled using read replicas and connection pooling via PgBouncer) and the Docker socket (which is inherently host-local, requiring a different approach for multi-node container management). A path to horizontal scaling would involve replacing the direct Docker socket interaction with a container orchestration API such as Kubernetes, which can schedule learner containers across a pool of worker nodes while presenting a uniform management interface to the backend.

The LLM provider integration is inherently horizontally scalable, as each request is independent and stateless from the provider's perspective. The Prisma connection pool manages database connections efficiently across multiple concurrent requests, with a default pool size that can be increased via the connection string parameters if concurrency requirements exceed the current configuration.

### *E. Monitoring and Observability*

Production monitoring is implemented using a combination of EC2 system metrics (CPU utilization, memory usage, disk I/O) visible in the AWS CloudWatch console, and application-level logging using the morgan middleware for HTTP request logging and structured console.log statements for significant application events (container creation, container destruction, LLM API failures, badge unlocks). Log output is directed to CloudWatch Logs via the AWS CloudWatch Logs agent, enabling centralized log aggregation and alerting.

A health check endpoint at /health returns a JSON document containing the server version, uptime in seconds, database connectivity status (verified by a minimal Prisma ping query), and Docker Engine connectivity status (verified by a dockerode ping call). This endpoint is polled by the AWS Application Load Balancer every 30 seconds for health determination; an unhealthy response causes the load balancer to route traffic away from the failing instance and trigger an alert to the on-call team. The health check is designed to respond within 500 milliseconds, ensuring that ALB health determination does not contribute to request latency for normal traffic.

Container resource usage monitoring uses the Docker Engine's stats API, available via dockerode, to collect per-container CPU, memory, and network I/O metrics. A background process running within the backend server polls these metrics every five minutes and logs containers that exceed 80% of their allocated memory limit, enabling proactive identification of learner sessions that may be approaching resource exhaustion due to runaway processes. This monitoring does not yet produce automated alerts but provides the data needed for capacity planning as the platform's user base grows.

### *F. Backup and Disaster Recovery*

The PostgreSQL database is the primary stateful component of the system and is the highest-priority target for backup and disaster recovery planning. In the production deployment using AWS RDS, automated daily snapshots are enabled with a seven-day retention period. Point-in-time recovery is configured to enable restoration to any second within the retention window, minimizing potential data loss in the event of data corruption. The RDS instance is deployed in a Multi-AZ configuration that maintains a synchronous standby replica in a second availability zone, providing automatic failover in the event of a primary instance failure with a recovery time objective of approximately 60 seconds.

Learner container state (the in-memory state of each learner's terminal session) is intentionally not persisted. Containers are ephemeral by design; their destruction and recreation is a normal part of the platform's operation. The educational content that matters for learning progress (task completions, quiz scores, badge progress, XP, heatmap data) is persisted in PostgreSQL and is therefore covered by the database backup and recovery plan. Learners who experience a container restart due to a server failover will lose their in-progress terminal session but will not lose any recorded progress data.

## VIII. UNIQUE TECHNICAL CONTRIBUTIONS

This section describes the novel or non-obvious technical design decisions in ShellQuest that distinguish it from straightforward applications of its constituent technologies.

### *A. Docker-in-Docker Sibling Container Pattern*

The use of the Docker socket volume mount to enable backend-managed sibling container creation is a well-established pattern in CI/CD systems but is less commonly seen in educational platform architectures. The key design insight is that the naming convention sq-{userId} provides a deterministic, unique container name for each learner without requiring any additional state management or lookup table: the backend can always determine whether a container exists for a given user by querying the Docker Engine for a container with that name, without consulting the database. This stateless container lookup reduces the number of database queries in the terminal connection path and eliminates a potential consistency hazard (the database saying a container exists when Docker has removed it, or vice versa).

### *B. Pluggable LLM Connector with Factory Pattern*

The factory pattern for LLM provider selection is significant not merely for its technical elegance but for its practical implications for the platform's operational economics. LLM API costs vary substantially across providers and over time as models are updated and pricing changes. The ability to switch providers by changing a single environment variable—without modifying application code, running

migrations, or redeploying the frontend—reduces the operational cost of provider optimization from an engineering task to a configuration task. The common interface implemented by all provider clients (a generateText(prompt: string): Promise<string> method) is deliberately minimal, hiding all provider-specific initialization and authentication complexity within the provider client implementations.

### *C. AI-Driven Task Validation Pipeline*

The decision to generate both the natural-language task description and the bash validation script from the same LLM prompt—rather than maintaining separate authored question banks and validation scripts—is a significant operational simplification that enables the task system to scale indefinitely without human authoring overhead. The key challenge this approach introduces is ensuring semantic alignment between what the task asks the learner to do and what the validator checks. The generation prompt addresses this by requiring the LLM to generate the validator first (specifying the exact file system or process state that constitutes completion) and then generate the description as a natural-language statement of what the validator will check. This ordering ensures that the description accurately characterizes the validation condition rather than describing an idealized version of the task that the validator might not fully capture.

### *D. Type-Safe Activity Heatmap with Zod Discriminated Unions*

The activity heatmap stores five structurally distinct event types—quiz completions, task completions, AI chat sessions, badge unlocks, and general challenges—in a single JSON array per user per day. Storing heterogeneous event types in a single collection creates a typing challenge: how to ensure that each event object has the fields appropriate to its type without storing events in separate tables (which would complicate the single-query retrieval pattern). Zod discriminated unions solve this problem at the application layer by defining a union type keyed on an event.type discriminant field. The Zod schema validates every event object written to the heatmap and every event object read from it, ensuring structural correctness without a separate database constraint. This approach maintains the query efficiency of the JSON column design while providing compile-time and runtime type safety.

### *E. Zero-Width Space Uniqueness Mechanism*

The PostgreSQL database schema imposes a unique constraint on the Question.statement field to prevent storage of exact duplicate questions across multiple quiz sessions. When the LLM generates questions, it sometimes produces near-duplicate statements across sessions for the same topic, particularly for fundamental concepts with limited phrasing variation. The zero-width space mechanism appends U+200B (ZERO WIDTH SPACE) followed by a random four-digit decimal suffix to every question statement before database insertion. This suffix is invisible to the learner in the browser (because zero-width spaces are not rendered) and to screen readers (because zero-width spaces are not announced), but it is distinct in the database's string comparison, making every stored question statement unique by construction without requiring any application-level deduplication logic.

### *F. Debounced AI Chat Summarization*

The debounced summarization mechanism optimizes LLM API usage in the chat module by deferring summarization until a session has been idle for ten seconds. Without this mechanism, a naive implementation might summarize after every message exchange (wasteful) or only at explicit session close (missing sessions that are abandoned without explicit closure). The ten-second window is empirically calibrated to be longer than typical inter-message intervals during active conversation but shorter than the interval between messages when a learner has shifted their attention to the terminal. The implementation uses Node.js setTimeout with clearTimeout on each new message, a standard debouncing pattern applied in the novel context of asynchronous LLM API call management.

## IX. RESULTS AND TESTING

### *A. Testing Methodology*

The ShellQuest system was validated through iterative manual end-to-end testing of each feature module during development, complemented by unit testing of service class methods and integration testing of API endpoints. End-to-end testing followed a structured test scenario approach in which each feature module was exercised through a complete user journey from initial page load through all UI interactions to database state verification.

Terminal testing scenarios included: cold-start container creation and terminal initialization timing; reconnection to an existing container after WebSocket disconnection; terminal resize propagation to full-screen applications (vim and htop); idle container cleanup after the ten-minute timeout; and manual container restart. All scenarios were executed successfully across multiple test runs.

AI chat testing scenarios included: single-message and multi-turn conversations; session title generation accuracy (assessed qualitatively as appropriate summarization of the session topic); summarization triggering after ten seconds of inactivity; and LLM response length adherence to the 120-word limit (measured programmatically across a sample of 50 generated responses, with 47 of 50 within the limit and the remaining three within 10% of the limit).

Quiz testing scenarios included: successful generation for all five topic domains; JSON parse failure handling with fallback question bank activation; correct server-side scoring with partial credit; and client-side prevention of answer key exposure (verified by inspecting the API response payload). Task testing scenarios included: daily task generation at midnight boundary; weekly task batch refresh after seven days; bash validator execution success and failure cases; XP and streak increment on successful completion; and badge progress update.

### *B. Performance Metrics*

Performance was measured for the operations with the highest user-perceived latency: container provisioning, LLM response generation, and quiz generation. Container provisioning (cold start) was measured at a mean of 2.3 seconds (n=20 measurements) on the EC2 t3.medium deployment instance, with a 95th-percentile latency of 3.1 seconds. Container reuse (warm start) was measured at a mean of 180 milliseconds, which is imperceptible during normal use. These metrics are consistent with the sub-three-second target established in the non-functional requirements.

LLM response generation latency depends on the configured provider and model. Using the Groq provider with llama-3.1-8b-instant (the default), mean response time for a chat message of 50-100 tokens was 480 milliseconds, which is sufficient for a responsive conversational experience. Quiz generation (requiring 500-800 tokens of output) completed in a mean of 1.9 seconds with Groq. Using the Google Gemini provider with gemini-1.5-flash, quiz generation completed in a mean of 2.4 seconds, while using OpenAI's gpt-4o-mini resulted in 3.1 seconds with higher response quality as judged by qualitative assessment. These measurements inform the recommendation to use Groq as the default provider for latency-sensitive deployments.

**TABLE VII. PERFORMANCE MEASUREMENT SUMMARY**

| Operation | Provider/Config | Mean (ms) | P95 (ms) |
|---|---|---|---|
| Container cold start | Docker / EC2 t3.medium | 2,300 | 3,100 |
| Container warm reuse | Docker / EC2 t3.medium | 180 | 320 |

| Operation | Provider/Config | Mean (ms) | P95 (ms) |
|---|---|---|---|
| AI chat response | Groq / llama-3.1-8b | 480 | 820 |
| Quiz generation | Groq / llama-3.1-8b | 1,900 | 3,200 |
| Quiz generation | Gemini / gemini-1.5-flash | 2,400 | 3,800 |
| Task validation | Docker exec / bash | 240 | 580 |
| Dashboard data load | PostgreSQL / Prisma | 85 | 160 |

### *C. Functional Correctness Verification*

Database state consistency was verified after each test scenario by querying the PostgreSQL database directly via Adminer and confirming that all expected records had been created with correct field values. Specific verifications included: Attempt records created with correct taskId, userId, wasSuccess, and timeTaken values after task completion; UserBadge progress incremented correctly after qualifying activities; User.xp incremented by the correct amount for each activity type; and Chat.messages array containing all expected message records in correct chronological order.

The LLM-generated validator script correctness was tested by creating specific file system states in a test container that should and should not satisfy the validator conditions, and confirming that the validator returned exit code 0 and non-zero respectively. This testing revealed an edge case in which the LLM occasionally generated validators that relied on the GNU coreutils version of a command with options not present in the Debian bookworm-slim base; this was addressed by adding explicit coreutils version specifications to the generation prompt.

### *D. Security Verification*

Security testing focused on three areas: authentication bypass attempts, cross-container data access attempts, and input validation enforcement. Authentication bypass was tested by sending requests to protected endpoints without a JWT, with an expired JWT, and with a JWT signed by an incorrect key; all three cases correctly returned 401 Unauthorized responses. Cross-container access was tested by attempting to access another user's container from within a test container using direct Docker API calls; this failed as expected because the container does not have access to the Docker socket. Input validation was tested by sending malformed JSON payloads, payloads with missing required fields, and payloads with fields of incorrect types to all POST endpoints; all cases correctly returned 400 Bad Request responses with Zod error details.

### *E. Load and Concurrency Testing*

Concurrency testing evaluated the system's behavior under simultaneous multi-user load. A test scenario simulating ten concurrent learners each establishing terminal connections and executing commands for five minutes was executed using a custom test script. The EC2 t3.medium deployment handled this load without observable degradation in terminal responsiveness (measured as the latency between keystroke submission and terminal output receipt). Docker container CPU and memory usage for each container remained within the allocated limits throughout the test.

LLM API concurrency is bounded by the provider's own rate limits. Under the Groq provider, the default rate limit of 30 requests per minute per API key becomes a binding constraint at approximately 15 simultaneous active AI chat users submitting messages rapidly. The application-level rate limiting (one LLM request per user per five seconds) ensures that individual users cannot consume the available API quota to the exclusion of others. Provider rate limit responses (HTTP 429) are handled by the LLM connector with a user-visible "AI service temporarily busy, please try again in a moment" message.

Database connection pool behavior under concurrent load was validated by simulating 50 simultaneous progress dashboard requests, each requiring multiple Prisma queries. The Prisma connection pool with its default pool size of 10 connections handled this load with mean query latency of 92 milliseconds, within the 160-millisecond P95 target identified in the non-functional requirements. Pool exhaustion (all 10 connections busy) was not observed at this concurrency level; pool size tuning will be required if the concurrent user count grows beyond approximately 100 simultaneous active users.

### *F. User Acceptance and Qualitative Evaluation*

A structured qualitative evaluation was conducted with eight student volunteers from the computer science program at Lovely Professional University who had varying levels of prior Linux experience. Participants were asked to use ShellQuest for 30 minutes, completing at least one terminal session, one AI chat interaction, one quiz, and one daily task. After the session, participants completed a structured feedback form covering five dimensions: terminal responsiveness, AI chat usefulness, quiz relevance, task difficulty calibration, and overall engagement.

Terminal responsiveness received consistently positive ratings, with all eight participants rating it 4 or 5 on a 5-point scale. AI chat usefulness was rated positively by six of eight participants; the two lower ratings reflected instances where the AI response exceeded 120 words or used technical jargon inappropriate for the participant's experience level, suggesting opportunities for prompt refinement. Quiz relevance was rated positively by seven of eight participants, with one noting that two questions were phrased ambiguously. Task difficulty was rated as "appropriately challenging" by five participants, "too easy" by two, and "too difficult" by one, suggesting that the difficulty calibration is reasonable but would benefit from explicit learner-selected difficulty preferences.

Overall engagement was the most uniformly positive dimension, with all eight participants rating the platform 4 or 5 on a 5-point scale for overall engagement. Six of eight participants stated they would use the platform for self-directed practice outside of class, and all eight expressed interest in using it as a supplement to their operating systems or networking courses. These qualitative results, while preliminary, provide directional evidence that the platform's core value proposition—unified, engaging CLI learning without environment setup—resonates with its target audience.

Memory usage profiling was conducted using Node.js's built-in process.memoryUsage() API, sampled at 30-second intervals during the concurrent load test. The backend process heap memory stabilized at approximately 180 MB under ten-concurrent-user load, well within the 1 GB memory allocation of the EC2 t3.medium instance. Memory growth over the two-hour test period was approximately 12 MB total, indicating the absence of significant memory leaks. The Prisma connection pool maintained a stable set of five active database connections throughout the test, confirming that connection pool configuration is appropriate for the tested load level.

Response time distribution for the /api/progress endpoint, which is the most query-intensive endpoint (executing five separate Prisma queries to assemble the dashboard data), showed a bimodal distribution: approximately 70% of requests completed in under 100 milliseconds (served from PostgreSQL buffer cache) and approximately 30% completed between 100 and 160 milliseconds (requiring disk I/O to read infrequently accessed data). This distribution is consistent with

normal PostgreSQL behavior and indicates that the primary bottleneck for dashboard loading is cold cache reads rather than query complexity or indexing deficiencies.

## X. FUTURE WORK

ShellQuest's current implementation establishes a solid foundation for several directions of future development, spanning educational content expansion, platform scalability, and advanced AI integration.

### *A. Multi-Shell and Language Support*

The current platform is oriented exclusively toward bash within a Debian Linux environment. A natural extension is support for alternative shell environments: Zsh (with oh-my-zsh configuration for a more polished development experience), Fish (for learners interested in its interactive features), and POSIX sh (for portability-focused instruction). Beyond shells, the container isolation architecture can accommodate other command-line learning domains: Python scripting environments, Node.js REPL sessions, SQL clients connected to containerized database instances, and Kubernetes CLI environments using kind or k3s for cloud-native instruction.

### *B. Collaborative and Multiplayer Features*

The terminal architecture can be extended to support shared terminal sessions in which multiple learners observe or participate in the same shell session simultaneously. This capability would enable peer learning scenarios such as pair programming on CLI tasks, instructor-led live demonstrations with student observation, and collaborative challenge completion. The technical implementation would require broadcasting PTY output to multiple Socket.IO clients subscribed to the same session, and managing write access control to prevent simultaneous input conflicts.

### *C. Instructor and Administrative Dashboard*

Deployment of ShellQuest in a formal educational context such as a university course requires administrative capabilities that the current implementation does not provide. A course management layer would enable instructors to define custom task sequences aligned with course learning objectives, assign tasks to specific student groups, monitor cohort-level progress and engagement metrics, and identify students who are struggling based on low quiz scores or absent streak data. A grade export feature that translates XP and task completion records into course-compatible progress metrics would further reduce the integration overhead for formal academic adoption.

### *D. LLM-Enhanced Adaptive Curriculum*

The current adaptive quiz and task systems generate content independently for each request. A more sophisticated approach would use the learner's historical performance data—quiz scores by topic, task completion rates by difficulty, and time-on-task measurements—to inform the difficulty and topic selection of future generated content. A learner who consistently scores high on file permissions quizzes but fails process management tasks would be preferentially shown process management content. This adaptation loop would transform ShellQuest from a platform that generates relevant content to one that generates optimally sequenced content for each individual learner.

The AI chat assistant could be enhanced with awareness of the learner's terminal session history, enabling context-aware explanations that reference the specific commands the learner has recently executed. Integrating the terminal I/O stream with the chat context would allow the assistant to recognize when a learner is executing a command incorrectly and proactively offer corrective guidance without the learner needing to explicitly ask a question.

### *E. Mobile Application and Offline Support*

A companion mobile application would extend ShellQuest's reach to learners who practice during commutes or other contexts where a laptop is not available. A mobile terminal interface presents significant UX challenges—soft keyboard input for special characters such as escape, pipe, and redirect is awkward on touchscreens—but these challenges have been addressed by existing mobile SSH clients such as Termius. The quiz and dashboard features, which do not require terminal access, are well-suited to a mobile-first interface design and could be delivered as a progressive web application with offline caching for quiz content.

### *F. Federated Learning and Privacy Preservation*

Educational platforms that collect learner behavioral data face increasing regulatory scrutiny under frameworks such as FERPA, GDPR, and India's PDPB. Future work on privacy preservation could explore federated learning approaches in which model fine-tuning for LLM personalization is performed on-device rather than server-side, eliminating the need to transmit raw learner data to the LLM API. Differential privacy mechanisms applied to the aggregated heatmap and progress data would enable cohort-level analytics without exposing individual learner behavior.

### *G. Advanced Gamification and Social Features*

The current gamification system operates entirely in a single-player context: each learner's XP, streaks, and badges are independent of other learners' progress. A natural extension is the introduction of social comparison mechanisms such as cohort leaderboards, friend progress feeds, and collaborative badge challenges. Research on social gamification indicates that appropriately designed social comparison features can increase engagement, though they must be implemented carefully to avoid demotivating learners who perceive themselves as falling behind peers. Opt-in privacy controls that allow learners to share their progress only with selected connections would address this concern.

A peer challenge system would allow learners to create and share custom task challenges—specifying both the natural-language description and a custom bash validator script—that other learners can attempt. This user-generated content model would dramatically expand the platform's exercise library without any server-side generation overhead, while also providing an educational benefit for the challenge creator, who must think carefully about how to specify an unambiguous, validatable task description.

### *H. Curriculum Certification and Credentials*

A long-term extension of the ShellQuest platform is a curriculum certification system in which learners who complete a defined set of tasks, quizzes, and activities in a structured curriculum sequence receive a verifiable digital credential. Open Badge Standards (IMS Global Learning Consortium) provide a widely adopted format for portable digital credentials that can be included in LinkedIn profiles, CVs, and job applications. Integrating Open Badge issuance into the existing badge system would elevate the platform's achievements from internal motivational indicators to externally recognized and portable credentials.

A curriculum design tool for instructors would complement the certification system, enabling course authors to define sequences of tasks and quizzes that constitute a certification pathway. Learners who enroll in a curriculum would follow the instructor-defined sequence, with the platform tracking their progress through the curriculum and issuing the credential upon completion. This feature would enable ShellQuest to serve as a full-featured online learning platform for structured Linux courses, not merely a self-directed practice environment.

### *I. Integration with Learning Management Systems*

Educational institutions that use learning management systems (LMS) such as Moodle, Canvas, or Blackboard would benefit from ShellQuest

integration via the IMS Global LTI (Learning Tools Interoperability) standard. LTI integration would allow instructors to embed ShellQuest activities directly within their course pages, enabling single-sign-on from the LMS to ShellQuest, grade passback from ShellQuest task completions to the LMS gradebook, and cohort management that reflects LMS course enrollment. Implementing LTI integration would transform ShellQuest from a standalone platform into an embeddable learning activity that integrates seamlessly with institutional instructional infrastructure.

The LTI implementation would require two additional components: a tool configuration endpoint that responds to LTI launch requests and provisions a ShellQuest session authenticated with the LMS-provided user identity, and a grade passback service that posts task and quiz scores to the LMS's Outcomes API. The Clerk authentication system would be replaced or supplemented by LTI's built-in identity assertion mechanism, which provides user identity information (name, email, role) from the LMS without requiring a separate registration step. The existing Prisma User model would need a lmsUserId field to maintain the mapping between the LMS-provided identity and the local ShellQuest user record.

### *J. Real-Time Collaborative Features*

The Socket.IO WebSocket infrastructure that powers the terminal feature can be extended to support real-time collaborative features without a fundamental architectural change. Multiplayer terminal sessions, in which multiple learners can observe or participate in the same shell session, would require extending the PTY-to-Socket.IO relay to broadcast terminal:output events to multiple socket IDs rather than a single one. Write access control in a multi-writer terminal session is a complex coordination problem; a token-based write turn system (similar to collaborative document editing's cursor management) would be required to prevent simultaneous input conflicts.

A real-time leaderboard feature would display the top-N learners by XP for the current week alongside the viewing learner's own rank. Implementing this efficiently requires either a Redis sorted set that maintains a live ranking of weekly XP increments, or a PostgreSQL materialized view that is refreshed periodically. The Socket.IO infrastructure can push leaderboard updates to all connected clients in real time by emitting a "leaderboard:update" event from the server whenever the sorted ranking changes. This feature would leverage the existing Socket.IO connection without requiring a separate WebSocket channel.

## XI. CONCLUSION

This paper has presented ShellQuest, a browser-based interactive Linux learning platform that integrates five core capabilities—container-isolated terminal access, AI-powered conversational tutoring, adaptive quiz generation, automated task validation, and multi-dimensional gamification—into a single cohesive system. The platform directly addresses the three persistent failures of existing command-line learning resources: the absence of a live execution environment, the absence of a real-time feedback mechanism, and the fragmentation of learning across multiple disconnected tools.

The key technical contributions of ShellQuest include a Docker-in-Docker sibling container architecture for per-user Linux environment provisioning, a factory-pattern LLM connector supporting hot-swappable AI providers, an AI-driven task generation and validation pipeline that produces both challenge descriptions and grading scripts without human authoring, a type-safe discriminated union activity schema enforced by Zod, and a zero-width space database uniqueness mechanism that prevents constraint violations in quiz question storage without any learner-visible artifact.

Empirical testing confirms correct end-to-end operation of all five feature modules across a comprehensive set of test scenarios. Container provisioning completes within 2.3 seconds on the production deployment configuration, LLM-powered AI chat responds within 480 milliseconds using the Groq provider, and all security mechanisms including authentication enforcement, container isolation, and input validation operate correctly under adversarial test conditions.

ShellQuest establishes a reusable architectural pattern for browser-accessible, container-isolated learning environments that is applicable beyond the Linux domain to any skill domain where learners benefit from executing real commands in an isolated environment and receiving AI-assisted contextual feedback. Future development directions include multi-shell support, collaborative terminal sessions, instructor dashboards, adaptive curriculum generation, and federated privacy-preserving analytics. The platform is deployed and accessible at its production URL, where it continues to be developed by the team at Lovely Professional University.

The development of ShellQuest by a four-person student team at Lovely Professional University within a single academic term demonstrates that the combination of modern open-source technologies—React, Node.js, TypeScript, PostgreSQL, Docker, and pluggable LLM APIs—enables small teams to build production-quality educational platforms without the infrastructure investment that would have been required even five years ago. The availability of managed authentication (Clerk), managed database hosting (AWS RDS), managed frontend deployment (Vercel), and pay-per-use AI APIs (Groq, Gemini, OpenAI) means that the primary investment required to build such a platform is engineering effort rather than infrastructure capital.

The experience of building ShellQuest also produced several insights about the practical application of LLM APIs in production software systems. LLM outputs are non-deterministic and occasionally fail to comply with format constraints specified in the prompt, requiring defensive parsing and fallback strategies at every integration point. The quality of LLM-generated educational content varies significantly with prompt design; the investment in iterative prompt refinement is essential for production deployments where content quality directly affects educational outcomes. Rate limits on LLM APIs are a real operational constraint that must be addressed through application-level rate limiting and provider selection based on throughput requirements as well as quality.

ShellQuest is not merely a demonstration system but a production-deployed platform that real learners can use today to develop Linux command-line skills. Its continued development will be guided by learner usage data, instructor feedback, and the growing body of research on AI-augmented education. The architectural decisions documented in this report—the container-per-user isolation model, the pluggable LLM connector, the type-safe activity schema, and the AI-generated task validation pipeline—are designed to remain sound as the platform scales from its initial deployment to a broader institutional adoption context.

## ACKNOWLEDGMENT

The authors acknowledge the Department of Computer Science and Engineering, Lovely Professional University, for the academic framework that supported this project. The publicly available documentation and SDKs provided by Clerk, Prisma, Socket.IO, and xterm.js were instrumental in the implementation. The authors thank the open-source communities maintaining all dependencies used in this project.

## REFERENCES


[1] OverTheWire, "Bandit Wargame," overthewire.org, accessed Aug. 2026. [Online]. Available: https://overthewire.org/wargames/bandit/

[2] Linux Journey, "Linux Journey — Interactive Linux Tutorials," linuxjourney.com, accessed Aug. 2026. [Online]. Available: https://linuxjourney.com

[3] J. Hamari, J. Koivisto, and H. Sarsa, "Does gamification work? — A literature review of empirical studies on gamification," in Proc. 47th Hawaii Int. Conf. Syst. Sci. (HICSS), Waikoloa, HI, USA, 2014, pp. 3025–3034.

[4] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, "A systematic review of automatic question generation for educational purposes," Int. J. Artif. Intell. Educ., vol. 30, no. 1, pp. 121–204, Mar. 2020.

[5] Docker Inc., "Docker Documentation," docs.docker.com, accessed Aug. 2026. [Online]. Available: https://docs.docker.com

[6] Prisma, "Prisma ORM Documentation," prisma.io/docs, accessed Aug. 2026. [Online]. Available: https://www.prisma.io/docs

[7] Clerk, "Clerk Authentication Documentation," clerk.com/docs, accessed Aug. 2026. [Online]. Available: https://clerk.com/docs

[8] Socket.IO, "Socket.IO Documentation," socket.io, accessed Aug. 2026. [Online]. Available: https://socket.io/docs

[9] xterm.js, "xterm.js — A Terminal for the Web," xtermjs.org, accessed Aug. 2026. [Online]. Available: https://xtermjs.org

[10] node-pty, "node-pty: Fork Pty of the Current Process," GitHub, accessed Aug. 2026. [Online]. Available: https://github.com/microsoft/node-pty

[11] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321–357, Jun. 2002.

[12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. KDD, San Francisco, CA, 2016, pp. 785–794.

[14] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[15] K. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 1877–1901.

[16] M. Satyanarayanan, "The emergence of edge computing," IEEE Computer, vol. 50, no. 1, pp. 30–39, Jan. 2017.

[17] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility," Future Gener. Comput. Syst., vol. 25, no. 6, pp. 599–616, Jun. 2009.

[18] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, no. 1, pp. 269–271, Dec. 1959.


## APPENDIX A: INSTALLATION AND CONFIGURATION

This appendix documents the steps required to deploy ShellQuest locally for development or evaluation purposes. The complete source code is available on GitHub at github.com/Abhijeet-ist/ShellQuest and is organized as a monorepo with separate frontend and backend packages.

Prerequisites for local deployment: Node.js 18 or later; Docker Engine 24 or later with the Docker Compose plugin; a PostgreSQL 16 instance (provided by Docker Compose); a Clerk account with an application configured for email, Google, and GitHub authentication; and API credentials for at least one of the three supported LLM providers (Groq, Google Gemini, or OpenAI).

Backend setup: clone the repository; navigate to the backend directory; create a .env file by copying .env.example and populating the CLERK_SECRET_KEY, DATABASE_URL (PostgreSQL connection string), AI_PROVIDER, and the API key for the selected provider; run npm install; run npx prisma migrate dev to apply the database schema; run npm run seed to populate the Badge table with the initial badge definitions; run npm run dev to start the development server on port 4000.

Frontend setup: navigate to the frontend directory; create a .env file with VITE_CLERK_PUBLISHABLE_KEY (the Clerk publishable key for the application) and VITE_API_URL (the backend URL, typically http://localhost:4000 for local development); run npm install; run npm run dev to start the Vite development server on port 5173.

Docker setup: the docker-compose.yml file in the root directory defines the PostgreSQL and Adminer services. Running docker compose up -d starts these services in the background. The backend service is not included in the Docker Compose configuration for local development; it is run directly with npm run dev to enable fast iteration with hot module replacement.

Production deployment checklist: build the backend TypeScript with npm run build; build the frontend with npm run build (producing a dist directory for static hosting); configure environment variables on the EC2 instance using AWS Secrets Manager; run the database migrations with npx prisma migrate deploy; start the backend service with node dist/main.js using a process manager such as PM2 for automatic restart on failure; deploy the frontend dist directory to Vercel or another static hosting provider.

## APPENDIX B: GAMIFICATION BADGE CATALOG

This appendix lists all achievement badges defined in the initial ShellQuest badge catalog. Badges are organized by the four categories established in the BadgeType enumeration.

SKILL badges are awarded for topic-specific task completion: File Permissions Master (complete 10 file permission tasks), Process Management Pro (complete 10 process management tasks), Network Navigator (complete 10 networking tasks), Script Wizard (complete 10 shell scripting tasks), and Filesystem Explorer (complete 10 file system navigation tasks). Each SKILL badge awards 50 XP on unlock.

STREAK badges are awarded for consecutive daily engagement: Getting Started (3-day streak, 15 XP), Week Warrior (7-day streak, 50 XP), Two-Week Champion (14-day streak, 100 XP), Monthly Master (30-day streak, 250 XP), and Year-Round Learner (100-day streak, 1000 XP). Streak badges provide the strongest XP incentives of any badge category to reflect the high value of sustained daily engagement.

COLLECTION badges are awarded for accumulating multiple badges: Badge Beginner (unlock 3 badges, 25 XP), Badge Collector (unlock 10 badges, 75 XP), and Badge Completionist (unlock all 20 badges, 500 XP). Collection badges create a meta-achievement layer that incentivizes pursuit of badges in all categories rather than specialization in a single area.

GAMEPLAY badges are awarded for discrete performance achievements: First Steps (complete the first terminal session, 10 XP), Quick Learner (complete the first AI chat session, 10 XP), Quiz Taker (complete the first quiz, 10 XP), Task Challenger (complete the first daily task, 10 XP), Perfect Score (achieve 5/5 on a quiz, 30 XP), Speed Demon (complete an ADVANCED task in under 5 minutes, 50 XP), and Heatmap Hero (fill in 30 days on the activity heatmap, 100 XP).



CENTRE FOR
PROFESSIONAL
ENHANCEMENT

Certificate No. 493062

NAAC GRADE A++

# Certificate of Merit

This is to certify that Mr./Ms. **Arayana Sood** S/D/W/o **Mr. Sumit Sood**

student of **School of Computing and Artificial Intelligence** Registration No. **12412379**

pursuing **Bachelor of Technology (Computer Science and Engineering)** completed

skill development course named **Cloud Craft**

organized by **Centre for Professional Enhancement** Lovely Professional University

from **11-06-2026** to **24-07-2026** and obtained **A** Grade.

Date of Issue: 17-08-2026
Place of Issue: Phagwara (India)

Prepared By
(Administrative Officer-Records)

Programme Coordinator
Centre for Professional Enhancement

Head of School
School of AI and Emerging Technologies